UNIVERSIDAD POLITÉCNICA DE SAN LUIS POTOSÍ

UNIVERSIDAD POLITÉCNICA DE

SAN LUIS POTOSÍ

CARRERA: Ingeniería en Tecnologías de la Información

MATERIA: CNO IV: Seguridad Informática

TRABAJO: Informe de Seguridad Ofensiva

Ojeda Camacho José Carlos / 181629

PROFESOR: Mtro. Servando López Contreras

Contenido

1. Alcance y Metodología 7
1.1. Objetivo 7
1.2. Alcance del entorno 7
1.3. Metodología de evaluación 7
1.4. Reglas del ejercicio y limitaciones 8
2. Resumen Ejecutivo 8
3. Inventario de Vulnerabilidades 9
4. Hallzgos 11
4.1 Hallazgo 01 (VULN-01): Exposición de Información Sensible vía robots.txt y Directory Listing (Reto 1) 11
4.1.1 Descripción técnica y causa raíz 11
4.1.2 Procedimiento de comprobación 12
4.1.3 Evidencia técnica 12
4.1.4 Impacto y riesgo 13
4.1.5 Mitigaciones 13
4.2 Hallazgo 02 (VULN-02): Exposición de Metadatos y Endpoints Internos vía Archivos de Configuración (Reto 2) 14
4.2.1 Descripción técnica y causa raíz 14
4.2.2 Procedimiento de comprobación 14
4.2.3 Evidencia técnica 15
4.2.4 Impacto y riesgo 16
4.2.5 Mitigaciones 16
4.3 Hallazgo 03 (VULN-03): Insecure Direct Object Reference (IDOR) en Consulta de Expedientes (Reto 3) 17
4.3.1 Descripción técnica y causa raíz 17
4.3.2 Procedimiento de comprobación 17
4.3.3 Evidencia técnica 18
4.3.4 Impacto y riesgo 19
4.3.5 Mitigaciones 20
4.4 Hallazgo 04 (VULN-04): Exposición de Código Fuente mediante Archivos de Respaldo Residuales (Reto 4) 20
4.4.1 Descripción técnica y causa raíz 20
4.4.2 Procedimiento de comprobación 21

4.4.3 Evidencia técnica 21
4.4.4 Impacto y riesgo 22
4.4.5 Mitigaciones 23
4.5 Hallazgo 05 (VULN-05): Exposición de Información por Manejo Inadecuado de Excepciones (Reto 5) 23
4.5.1 Descripción técnica y causa raíz 23
4.5.2 Procedimiento de comprobación 24
4.5.3 Evidencia técnica 24
4.5.4 Impacto y riesgo 25
4.5.5 Mitigaciones 26
4.6 Hallazgo 06 (VULN-06): Escalada de Privilegios Vertical por Confianza en Datos del Cliente (Reto 6) 26
4.6.1 Descripción técnica y causa raíz 26
4.6.2 Procedimiento de comprobación 27
4.6.3 Evidencia técnica 27
4.6.4 Impacto y riesgo 29
4.6.5 Mitigaciones 29
4.7 Hallazgo 07 (VULN-07): Inyección SQL Ciega (Blind SQLi) Basada en Booleanos (Reto 7) 30
4.7.1 Descripción técnica y causa raíz 30
4.7.2 Procedimiento de comprobación 30
4.7.3 Evidencia técnica 31
4.7.4 Impacto y riesgo 32
4.7.5 Mitigaciones 32
4.8 Hallazgo 08 (VULN-08): Inyección de Contenido y Manipulación de Metadatos en Módulo de Reportes (Reto 8) 33
4.8.1 Descripción técnica y causa raíz 33
4.8.2 Procedimiento de comprobación 33
4.8.3 Evidencia técnica 34
4.8.4 Impacto y riesgo 35
4.8.5 Mitigaciones 35
4.9 Hallazgo 09 (VULN-09): Vulnerabilidad de Recorrido de Directorios (Path / Directory Traversal) en Descarga de Archivos (Reto 9) 36
4.9.1 Descripción técnica y causa raíz 36

4.9.2 Procedimiento de comprobación 36
4.9.3 Evidencia técnica 37
4.9.4 Impacto y riesgo 38
4.9.5 Mitigaciones 38
4.10 Hallazgo 10 (VULN-10): Vulnerabilidad de Secuestro de Sesión por Fijación de Sesión (Reto 10) 39
4.10.1 Descripción técnica y causa raíz 39
4.10.2 Procedimiento de comprobación 39
4.10.3 Evidencia técnica 40
4.10.4 Impacto y riesgo 41
4.10.5 Mitigaciones 41
4.11 Hallazgo 11 (VULN-11): Autorización Rota y Exposición de Historial de Servicios por IDOR (Reto 11) 42
4.11.1 Descripción técnica y causa raíz 42
4.11.2 Procedimiento de comprobación 42
4.11.3 Evidencia técnica 43
4.11.4 Impacto y riesgo 44
4.11.5 Mitigaciones 45
4.12 Hallazgo 12 (VULN-12): Autorización Rota y Exposición de Token de Validación en Cabeceras HTTP por IDOR (Reto 12) 45
4.12.1 Descripción técnica y causa raíz 45
4.12.2 Procedimiento de comprobación 45
4.12.3 Evidencia técnica 46
4.12.4 Impacto y riesgo 47
4.12.5 Mitigaciones 47
4.13 Hallazgo 13 (VULN-13): Manipulación de Parámetros de Negocio (*Parameter Tampering*) en Pasarela de Pago (Reto 13) 48
4.13.1 Descripción técnica y causa raíz 48
4.13.2 Procedimiento de comprobación 48
4.13.3 Evidencia técnica 49
4.13.4 Impacto y riesgo 50
4.13.5 Mitigaciones 50
4.14 Hallazgo 14 (VULN-14): Lógica de Validación Inconsistente en Vínculos de Proveedores y Contratos (Reto 14) 51

4.14.1 Descripción técnica y causa raíz 51
4.14.2 Procedimiento de comprobación 51
4.14.3 Evidencia técnica 52
4.14.4 Impacto y riesgo 53
4.14.5 Mitigaciones 53
4.15 Hallazgo 15 (VULN-15): Exposición de Metadatos y Restricción de Acceso en el Gateway Operacional (Reto 15) 54
4.15.1 Descripción técnica y causa raíz 54
4.15.2 Procedimiento de comprobación 54
4.15.3 Evidencia técnica 55
4.15.4 Impacto y riesgo 55
4.15.5 Mitigaciones 55
4.16 Hallazgo 16 (VULN-16): Falsificación de Cabeceras HTTP (*IP Spoofing*) para Eludir Restricciones en el Gateway Operacional (Reto 16) 56
4.16.1 Descripción técnica y causa raíz 56
4.16.2 Procedimiento de comprobación 56
4.16.3 Evidencia técnica 57
4.16.4 Impacto y riesgo 58
4.16.5 Mitigaciones 58
4.17 Hallazgo 17 (VULN-17): Confianza en Cabeceras de Red y Exposición de Datos de Telemetría Operacional (Reto 17) 59
4.17.1 Descripción técnica y causa raíz 59
4.17.2 Procedimiento de comprobación 59
4.17.3 Evidencia técnica 60
4.17.4 Impacto y riesgo 60
4.17.5 Mitigaciones 61
4.18 Hallazgo 18 (VULN-18): Descifrado de Hash SHA-1 y Bypass de Autenticación en Actualización de Firmware (Reto 18) 61
4.18.1 Descripción técnica y causa raíz 61
4.18.2 Procedimiento de comprobación 62
4.18.3 Evidencia técnica 62
4.18.4 Impacto y riesgo 64
4.18.5 Mitigaciones 64

4.19 Hallazgo 19 (VULN-19): Violación de Fronteras de Dependencia y Exposición de Topología en Contextos de Activos (Reto 19) 65
4.19.1 Descripción técnica y causa raíz 65
4.19.2 Procedimiento de comprobación 65
4.19.3 Evidencia técnica 66
4.19.4 Impacto y riesgo 67
4.19.5 Mitigaciones 67
4.20 Hallazgo 20 (VULN-20): Bypass de Validación de Estado Operacional e Interrupción de Válvulas Críticas (Reto 20) 68
4.20.1 Descripción técnica y causa raíz 68
4.20.2 Procedimiento de comprobación 68
4.20.3 Evidencia técnica 69
4.20.4 Impacto y riesgo 69
4.20.5 Mitigaciones 70
5. Cadena de Ataque y Correlación de Hallazgos 70
Fases de la Cadena de Ataque en INTERAFAS-LAB 70
Reconocimiento y Exposición Inicial (Fase Externa): 70
Acceso Inicial y Exfiltración de Datos (Fase de Compromiso Web): 71
Escalada de Privilegios y Manipulación Transaccional (Fase de Consolidación): 71
Paso de la Red Corporativa (IT) a la Red Industrial (OT): 71
Compromiso Final y Control de Actuadores (Fase Operacional Crítica): 72
6. Calidad del Informe, Conclusiones y Cierre 72
6.1 Conclusiones Propias de la Auditoría 72
6.2 Cierre Documental 72

1. Alcance y Metodología

1.1. Objetivo

El objetivo principal de esta evaluación de seguridad ofensiva (Red Teaming) es identificar, comprobar y documentar sistemáticamente las vulnerabilidades presentes en el entorno tecnológico de INTERAFAS. Adicionalmente, se busca evaluar de forma integral la resiliencia del sistema ante ciberataques dirigidos, analizando el impacto escalonado de una cadena progresiva de incidentes sobre la confidencialidad, la integridad y, de manera primordial, la disponibilidad operativa de los procesos físicos simulados de la organización.

1.2. Alcance del entorno

El alcance de este ejercicio se limitó de manera estricta al entorno académico controlado denominado INTERAFAS-LAB, desplegado y ejecutado a través de un servidor local (localhost:8080). La evaluación perimetral e interna abarcó exclusivamente los siguientes activos:

- Las aplicaciones web de gestión, administración y módulos de reportes.
- Las interfaces humano-máquina (HMI) utilizadas por los operadores de mantenimiento.
- Los canales de despliegue y actualización de firmware del *gateway* perimetral RTU-GW-07.
- La simulación interactiva de los componentes operacionales de la red metropolitana de abastecimiento, incluyendo la auditoría indirecta de los Controladores Lógicos Programables (PLC) de la estación Saint Louis, equipos de bombeo primario, red de distribución y válvulas asociadas a las zonas de servicio.

1.3. Metodología de evaluación

Para el desarrollo de las pruebas de penetración y el modelado de amenazas, se emplearon metodologías y marcos de referencia reconocidos a nivel internacional en el ámbito de la ciberseguridad industrial y corporativa:

- MITRE ATT&CK for ICS: Utilizado para categorizar las tácticas y técnicas de compromiso operacional ejecutadas, tales como la Modificación de Parámetros (T0836) y Evasión (T0840).
- OWASP (Open Worldwide Application Security Project): Empleado como estándar normativo para la evaluación de la seguridad web, enfocando el análisis en la correcta validación de la Autorización, la criptografía y el cumplimiento del Principio de Mínimo Privilegio (Broken Access Control).
- NIST SP 800-82 Rev. 3: Sus directrices para la seguridad en Sistemas de Control Industrial (ICS) guiaron el análisis de impacto y la formulación de las mitigaciones propuestas.

1.4. Reglas del ejercicio y limitaciones

Las reglas de enfrentamiento (*Rules of Engagement*) dictaminaron la prohibición estricta del uso de herramientas automatizadas de escaneo de vulnerabilidades o técnicas de explotación destructiva (como ataques volumétricos de denegación de servicio) que pudieran alterar el entorno fuera de los parámetros métricos del laboratorio.

Como limitación técnica, el ejercicio se restringió a la ejecución de técnicas manuales de explotación, manipulación directa de peticiones HTTP en consola y la ingeniería inversa de parámetros operacionales. Para las actividades de descifrado criptográfico, se limitó al uso de ataques de fuerza bruta y diccionarios en un entorno aislado *offline* (utilizando Hashcat para contraseñas de cuentas heredadas), garantizando en todo momento la estabilidad de la simulación.

2. Resumen Ejecutivo

La evaluación de seguridad ofensiva realizada sobre la infraestructura crítica de INTERAFAS revela una postura de ciberseguridad altamente deficiente y frágil. La organización presenta carencias sistémicas en la implementación de controles fundamentales, tales como la gestión robusta de identidades, la validación de integridad criptográfica en procesos de actualización y, de manera crítica, la segregación de privilegios dentro de su entorno de tecnologías de operación (OT). A diferencia de vulnerabilidades aisladas, los hallazgos demostraron que los defectos de diseño de INTERAFAS convergen, permitiendo a un atacante encadenar fallos lógicos para transicionar desde un compromiso a nivel de aplicación hasta la disrupción total del proceso físico industrial.

Durante el ejercicio, se logró mapear y ejecutar una cadena de ataque (*Kill Chain*) completa que evolucionó de forma progresiva. El incidente tuvo como vector de entrada inicial la recolección de información pasiva y el descubrimiento de una credencial de mantenimiento heredada (soporte.ot), cuyo hash SHA-1 fue expuesto por negligencia en el manifiesto público del canal de firmware. A partir de este hallazgo, la ausencia de firmas digitales para validar la autenticidad de los paquetes de actualización permitió la inyección de cargas útiles manipuladas y la escalada de privilegios en el HMI. Esta brecha inicial se agravó drásticamente debido a un control de acceso quebrantado (*Broken Access Control*), el cual violaba el principio de mínimo privilegio al permitir que un perfil de mantenimiento de bajo nivel tuviera visibilidad topológica completa de la red de distribución OT.

La crisis alcanzó su punto máximo cuando el atacante, habiendo realizado un reconocimiento exhaustivo de la cadena de dependencias (desde el gateway hasta las válvulas de campo), aprovechó la falta de límites lógicos de seguridad para inyectar una configuración de firmware alterada. La modificación maliciosa del parámetro de control operacional basal (pressure_setpoint_bar) desencadenó una cascada de fallas físicas incontrolables: un aumento de presión anómalo, la propagación de alarmas críticas y la caída irreversible de la disponibilidad operativa, llevando la simulación a un colapso clasificado como "CATASTROPHIC STATE". La conclusión primordial de esta auditoría es que INTERAFAS requiere una intervención de gobernanza urgente. No basta con la aplicación de parches de software

aislados; es imperativo rediseñar la arquitectura bajo un modelo de Defensa en Profundidad (*Defense-in-Depth*), separar lógicamente los entornos IT y OT, y establecer un Sistema de Gestión de Seguridad de la Información (SGSI) alineado con ISO/IEC 27001.

3. Inventario de Vulnerabilidades

El presente apartado consolida las vulnerabilidades descubiertas y explotadas durante el desarrollo de la auditoría en la plataforma INTERAFAS-LAB. Tal como lo estipula la metodología del ejercicio, los hallazgos no se perciben como eventos aislados, sino como eslabones de una cadena de compromiso mayor. A continuación se expone la tabla maestra que funge como inventario técnico central. En ella se detalla el identificador único de cada reto o vulnerabilidad, el componente lógico o físico que fue comprometido, la categoría técnica del riesgo (basada en el vocabulario de OWASP y MITRE ATT&CK for ICS) y el resultado o estado final que demuestra la materialización de la amenaza tras su exitosa explotación.

ID del Hallazgo	Clave de Referencia	Nombre del Reto / Vulnerabilidad	Categoría Principal / Marco de Referencia	Nivel de Riesgo / Impacto
01	**VULN-01**	**Exposición de Información Sensible vía robots.txt y Directory Listing**	**OWASP - Security Misconfiguration**	**Medio / Alto**
02	**VULN-02**	**Exposición de Metadatos y Endpoints Internos vía Archivos de Configuración**	**OWASP - Sensitive Data Exposure**	**Medio**
03	**VULN-03**	**Insecure Direct Object Reference (IDOR) en Consulta de Expedientes**	**OWASP - Broken Access Control**	**Crítico**
04	**VULN-04**	**Exposición de Código Fuente mediante Archivos de Respaldo Residuales**	**OWASP - Security Misconfiguration**	**Alto**
05	**VULN-05**	**Exposición de Información por Manejo Inadecuado de Excepciones**	**OWASP - Security Misconfiguration**	**Medio**

06	**VULN-06**	**Escalada de Privilegios Vertical por Confianza en Datos del Cliente**	**OWASP - Broken Access Control**	**Crítico**
07	**VULN-07**	**Inyección SQL Ciega (Blind SQLi) Basada en Booleanos**	**OWASP - Injection**	**Crítico**
08	**VULN-08**	**Inyección de Contenido y Manipulación de Metadatos en Módulo de Reportes**	**OWASP - Injection / XSS**	**Medio / Alto**
09	**VULN-09**	**Vulnerabilidad de Recorrido de Directorios (Path Traversal)**	**OWASP - Broken Access Control**	**Crítico**
10	**VULN-10**	**Secuestro de Sesión por Fijación de Sesión (Session Fixation)**	**OWASP - Broken Authentication**	**Alto**
11	**VULN-11**	**Autorización Rota y Exposición de Historial de Servicios por IDOR (API)**	**OWASP - Broken Access Control (BOLA)**	**Crítico**
12	**VULN-12**	**Autorización Rota y Exposición de Token de Validación en Cabeceras HTTP**	**OWASP - Broken Access Control**	**Alto**
13	**VULN-13**	**Manipulación de Parámetros de Negocio (*Parameter Tampering*) en Pagos**	**OWASP - Business Logic / Validation**	**Crítico**
14	**VULN-14**	**Lógica de Validación Inconsistente en Vínculos de Proveedores y Contratos**	**OWASP - Business Logic Errors**	**Medio**
15	**VULN-15**	**Exposición de Metadatos y Restricción de Acceso en Gateway Operacional**	**ICS/SCADA - Security Misconfiguration**	**Alto**

16	**VULN-16**	**Falsificación de Cabeceras HTTP (*IP Spoofing*) en Gateway Operacional**	**ICS/SCADA - Network Trust Flaw**	**Crítico**
17	**VULN-17**	**Confianza en Cabeceras de Red y Exposición de Datos de Telemetría**	**ICS/SCADA - Broken Access Control**	**Alto**
18	**VULN-18**	**Descifrado de Hash SHA-1 y Bypass en Actualización de Firmware**	**ICS/SCADA - Cryptographic / Auth Failure**	**Crítico**
19	**VULN-19**	**Violación de Fronteras de Dependencia y Exposición de Topología OT**	**ICS/SCADA - Topology Disclosure**	**Alto**
20	**VULN-20**	**Bypass de Validación Operacional e Interrupción de Válvulas Críticas**	**ICS/SCADA - Unsafe Actuator Control**	**Crítico**

4. Hallzgos

4.1 Hallazgo 01 (VULN-01): Exposición de Información Sensible vía robots.txt y Directory Listing (Reto 1)

4.1.1 Descripción técnica y causa raíz

Durante la fase de reconocimiento, se detectó una vulnerabilidad de exposición de información confidencial. El servidor revelaba la existencia de una ruta oculta de despliegue a través del archivo robots.txt. Al acceder a dicha ruta, el servidor web presentaba la vulnerabilidad de *Directory Listing* (listado de directorios), exponiendo archivos comprimidos (.zip) de publicación. Al descargar y extraer este paquete, se logró acceder a archivos de configuración de entorno (.env) que contenían información crítica y la primera bandera del ejercicio.

La vulnerabilidad es producto de una configuración deficiente del servidor web Apache (OWASP - *Security Misconfiguration*). Los administradores dejaron habilitada la directiva de auto-indexación (Options +Indexes) y cometieron el error de utilizar el archivo público robots.txt para intentar ocultar rutas sensibles, práctica ineficaz conocida como "seguridad por oscuridad".

4.1.2 Procedimiento de comprobación

El vector de ataque se validó de manera exitosa mediante la siguiente secuencia metodológica manual:

1. Se inspeccionó la raíz del servidor web accediendo mediante el navegador a la ruta http://localhost:8080/robots.txt.
2. Dentro de la respuesta en texto plano, se identificó la regla Disallow apuntando al directorio oculto /assets/docs/public/revision/.
3. Al introducir dicha URL directamente en la barra de direcciones, se confirmó que el listado de directorios estaba activo (Index of /assets/docs/public/revision), revelando diversos archivos de mantenimiento.
4. Se procedió a descargar el archivo de empaquetado del sistema nombrado portal-release_20260407_rc3.zip.
5. Se extrajo el contenido de manera local en el equipo de auditoría, encontrando internamente carpetas de despliegue (deploy) que albergaban archivos de configuración y variables de entorno (como release-check.env), donde se localizó en texto claro la bandera de validación correspondiente al reto.

4.1.3 Evidencia técnica

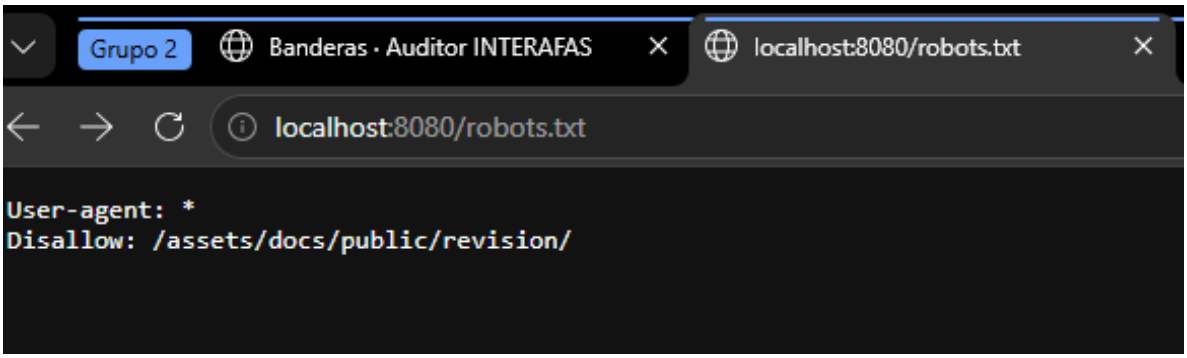


Figura 1. *Consulta directa al archivo público robots.txt desde el navegador, evidenciando la regla "Disallow" que expone la ruta restringida del directorio de revisiones del sistema.*

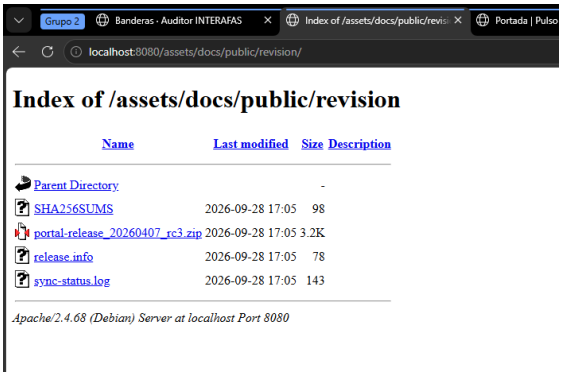


Figura 2. *Vulnerabilidad de Directory Listing habilitada en la configuración del servidor Apache, revelando la estructura interna y permitiendo la descarga directa del paquete de despliegue comprimido portal-release.*

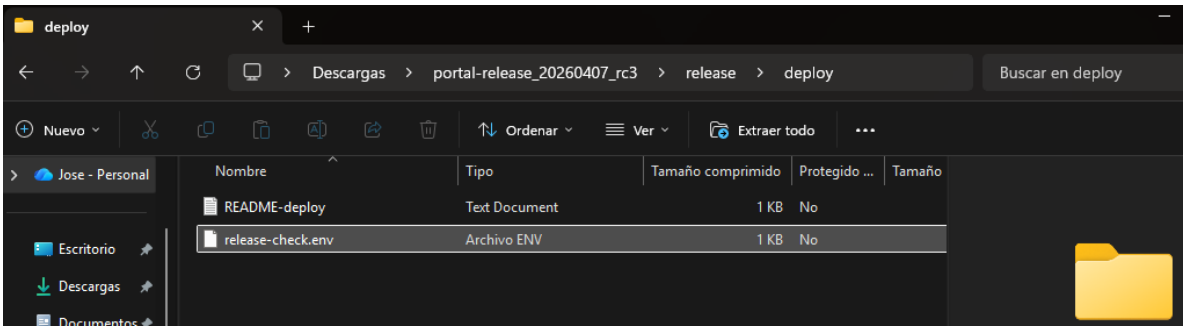


Figura 3. *Extracción local del paquete de despliegue descargado y localización del archivo de variables de entorno (.env) que albergaba información sensible y la bandera del reto.*

4.1.4 Impacto y riesgo

La explotación de esta vulnerabilidad representa un impacto directo y sumamente crítico sobre la Confidencialidad de la organización, sentando además las bases para comprometer la Integridad del sistema en fases posteriores. La exposición pública de paquetes de despliegue, archivos de inicialización y ficheros .env suele derivar en la filtración masiva de credenciales de bases de datos, tokens de acceso a APIs de terceros, llaves criptográficas y la lógica interna estructurada del código fuente de la aplicación.

A nivel operativo e institucional, el riesgo es inminente. El material expuesto en estos directorios mal configurados actúa frecuentemente como el escalón o punto de apoyo primario (*foothold*) que requiere un atacante para perfilar la tecnología subyacente. Al contar con los secretos de entorno, un adversario elude por completo las barreras del perímetro externo de seguridad, permitiéndole autenticarse en servicios internos, secuestrar infraestructuras de *backend* o escalar privilegios sin necesidad de desarrollar *exploits* complejos, lo que agrava sustancialmente la superficie de ataque de INTERAFAS.

4.1.5 Mitigaciones

Para remediar la causa raíz de este hallazgo, se recomiendan las siguientes medidas correctivas:

1. **Deshabilitar Directory Listing:** Configurar el servidor Apache para denegar el listado automático de archivos agregando o modificando la directiva a Options -Indexes en el archivo principal de configuración o mediante un archivo .htaccess.
2. **Políticas de despliegue seguro:** Evitar empaquetar o almacenar archivos comprimidos (.zip, .bak), código fuente o archivos de entorno (.env) dentro de la raíz de documentos públicos (DocumentRoot) del servidor web.
3. **Control de Acceso Real:** No depender del archivo robots.txt para proteger información. Los directorios de revisión, despliegue o administración deben estar protegidos mediante autenticación y controles de acceso estrictos a nivel de servidor.

4.2 Hallazgo 02 (VULN-02): Exposición de Metadatos y Endpoints Internos vía Archivos de Configuración (Reto 2)

4.2.1 Descripción técnica y causa raíz

Durante la fase de reconocimiento de la aplicación web, se identificó una vulnerabilidad de exposición de información sensible y metadatos del sistema. Al inspeccionar el código fuente del lado del cliente (Frontend), se descubrió que la aplicación dependía de un archivo JavaScript de configuración (runtime-config.js) que se cargaba de forma pública. Este archivo declaraba de manera explícita un diccionario con rutas y *endpoints* internos de la API y de los manifiestos de construcción (*build manifests*) que no estaban visibles en la navegación normal del sitio. Al acceder a uno de estos *endpoints* ocultos, el servidor devolvió un archivo JSON con metadatos de calidad (QA) y validaciones de la versión, exponiendo la segunda bandera del ejercicio.

La causa raíz de esta vulnerabilidad recae en una deficiencia en la gestión del ciclo de vida de desarrollo de software (OWASP - *Security Misconfiguration* y *Sensitive Data Exposure*). Los desarrolladores dejaron artefactos de depuración, marcadores de control de calidad (QA) y manifiestos de construcción accesibles en el entorno de producción. Además, el diseño del *frontend* expone la estructura interna de la API a cualquier usuario no autenticado, facilitando la fase de enumeración.

4.2.2 Procedimiento de comprobación

El vector de ataque se validó de manera exitosa mediante la siguiente secuencia metodológica manual:

1. Se inspeccionó el código fuente HTML de la página principal (index.html) utilizando las herramientas de desarrollador (F12) del navegador.
2. Al final del documento, se identificó la inclusión de *scripts* del sistema, destacando específicamente la ruta /assets/runtime-config.js.
3. Se accedió directamente a dicho archivo JavaScript desde el navegador, lo que reveló un objeto de configuración (window.INTERAFAS_RUNTIME) que contenía rutas internas como /api/scenario-state.php y /assets/meta/build-manifest.php.
4. Se navegó hacia la ruta interna descubierta (http://localhost:8080/assets/meta/build-manifest.php), la cual no contaba con controles de acceso.
5. El servidor respondió con un objeto JSON mostrando información de la versión, la rama de producción y el nodo quality_assurance, dentro del cual se encontraba en texto claro el *token* de validación (bandera) del reto.

4.2.3 Evidencia técnica

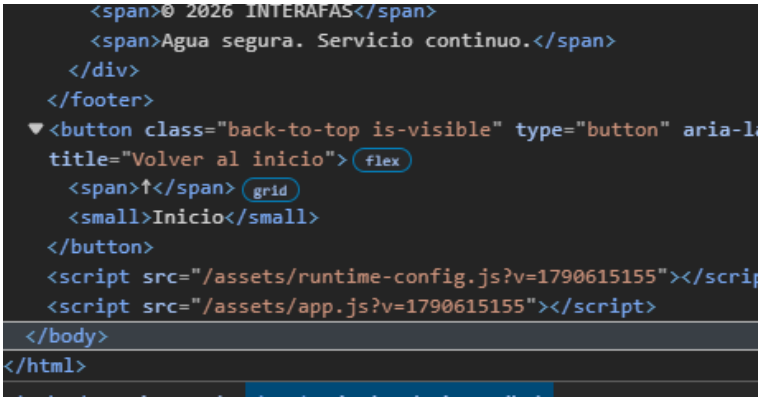


Figura 4. *Inspección del código fuente del lado del cliente revelando la carga de archivos de configuración estáticos y dependencias de la aplicación web.*

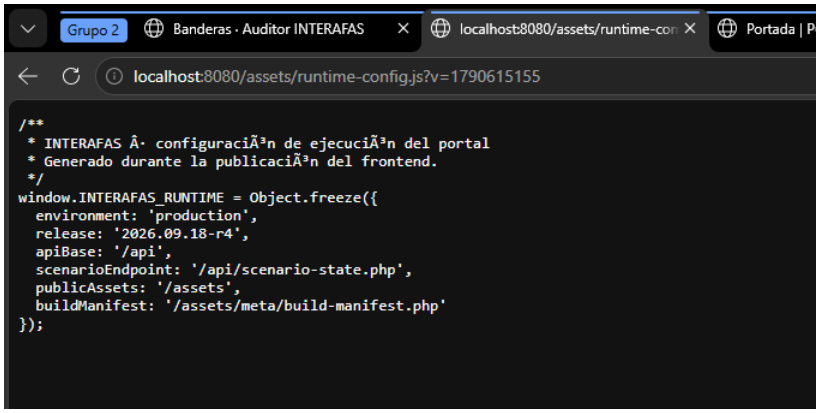


Figura 5. *Acceso directo al script de configuración del entorno, el cual expone un diccionario con las rutas de las APIs internas y los manifiestos del sistema.*

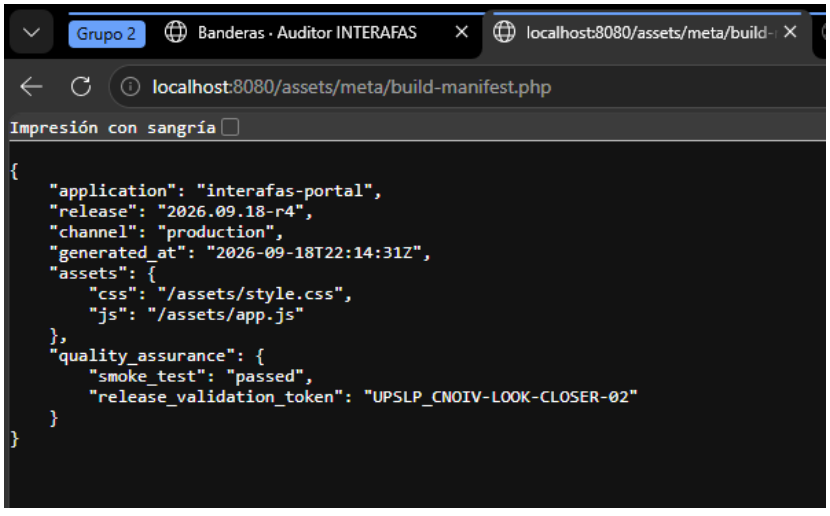


Figura 6. *Respuesta del servidor al consultar el endpoint interno de metadatos, revelando información del entorno de pruebas (QA) y el token de validación expuesto.*

4.2.4 Impacto y riesgo

La explotación de esta vulnerabilidad impacta negativamente la Confidencialidad de la aplicación, al revelar detalles precisos sobre la arquitectura interna, las versiones desplegadas y los *endpoints* ocultos de la API administrativa. Aunque no otorga acceso interactivo por sí misma, la fuga de metadatos de despliegue y validación reduce drásticamente el esfuerzo y el tiempo que necesita un atacante para mapear la superficie de ataque del sistema.

A nivel de riesgo operativo, esta fuga de información actúa como un catalizador para ataques secundarios. Al conocer las rutas exactas de las APIs, un adversario puede dirigir escaneos automatizados, ataques de fuerza bruta o inyecciones directamente contra *endpoints* que los administradores asumen como "ocultos" o "seguros". Dejar artefactos de control de calidad (QA) en producción evidencia fallos en los conductos de integración continua (CI/CD), lo que incrementa el riesgo de que en futuros despliegues se filtren credenciales o configuraciones críticas de bases de datos de la misma manera.

4.2.5 Mitigaciones

Para remediar la causa raíz de este hallazgo, se recomiendan las siguientes medidas correctivas:

1. **Limpieza de artefactos en producción:** Configurar los *pipelines* de integración y despliegue continuo (CI/CD) para que eliminen automáticamente directorios de metadatos (/meta), pruebas de humo (*smoke tests*) y manifiestos de construcción antes de mover el código al servidor web público.
2. **Control de Acceso en APIs:** Implementar validación de sesiones y controles de autorización en todos los *endpoints* que expongan el estado del sistema o metadatos de configuración (como /api/scenario-state.php o rutas de manifiestos), asegurando que solo usuarios administradores puedan consultarlos.
3. **Ofuscación y minimización:** Evitar codificar en duro (*hardcode*) diccionarios completos con las rutas del *backend* en los archivos JavaScript del cliente si estas rutas no son requeridas por el usuario anónimo. De ser necesarias, ofuscar el código puede añadir una capa extra de complejidad (aunque no reemplaza los controles de seguridad en el servidor).

4.3 Hallazgo 03 (VULN-03): Insecure Direct Object Reference (IDOR) en Consulta de Expedientes (Reto 3)

4.3.1 Descripción técnica y causa raíz

Durante la fase de auditoría de la aplicación web, se detectó una vulnerabilidad crítica de control de acceso clasificada como Insecure Direct Object Reference (IDOR), la cual pertenece a la categoría superior de *Broken Access Control* en el marco de OWASP. Al analizar el comportamiento del portal en la sección de consulta de expedientes ciudadanos, se observó que la aplicación no cargaba la información por defecto y devolvía un error de recurso no encontrado. Sin embargo, al inyectar explícitamente el parámetro id en la URL, el servidor devolvía la información solicitada sin verificar los privilegios de la sesión.

La causa raíz de este hallazgo radica en una deficiencia severa en la validación de autorización del lado del servidor. La aplicación confía ciegamente en la entrada proporcionada por el usuario (el parámetro numérico de la solicitud GET) para consultar la base de datos y construir la respuesta, omitiendo el paso lógico y crucial de verificar si la sesión activa (o la falta de ella) tiene los permisos adecuados para acceder a ese registro específico.

4.3.2 Procedimiento de comprobación

El vector de ataque se validó de manera exitosa mediante la siguiente secuencia metodológica manual:

1. Se intentó acceder a la ruta /expediente.php directamente, lo cual resultó en una respuesta HTTP 404 Not Found registrada en las herramientas de red, sugiriendo la falta de un identificador de recurso.
2. Haciendo uso del análisis de parámetros comunes, se modificó la URL agregando la cadena de consulta ?id=1.
3. El servidor respondió con un código HTTP 200 OK, y la página cargó exitosamente el expediente administrativo con los datos personales de la usuaria "Mariana Torres".
4. Para confirmar la ausencia total de controles de autorización, se manipuló nuevamente la URL iterando el identificador a ?id=2.
5. El sistema procesó la solicitud sin requerir autenticación y expuso el expediente de un segundo ciudadano ("Carlos Rivera"). Al final de este documento, en el apartado de observaciones, se localizó en texto claro la referencia de validación correspondiente al reto.

4.3.3 Evidencia técnica

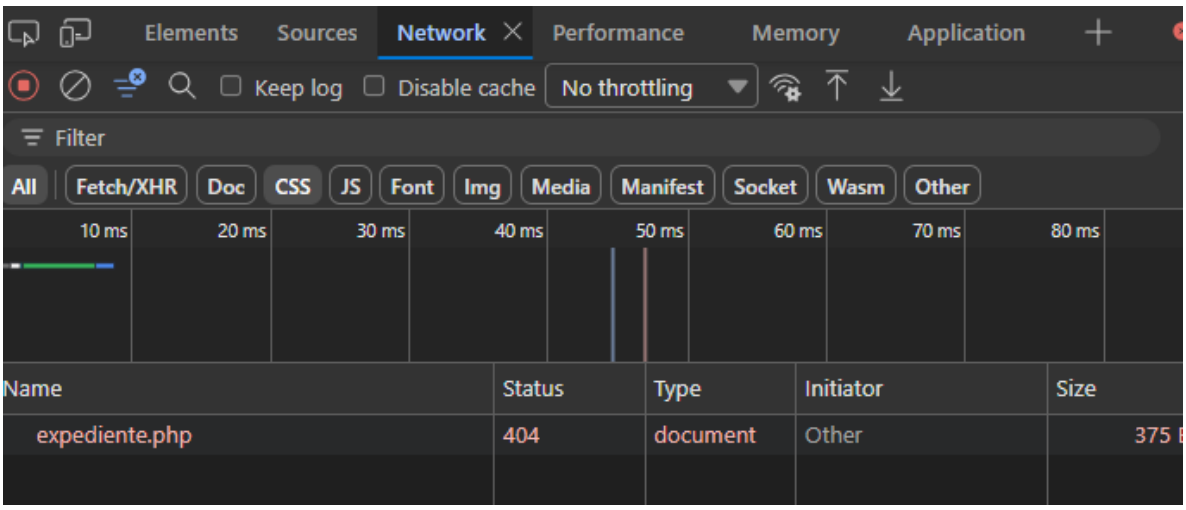


Figura 7. *Petición inicial al recurso de expedientes sin parámetros, resultando en un error HTTP 404 por la ausencia de un identificador en la cadena de búsqueda.*

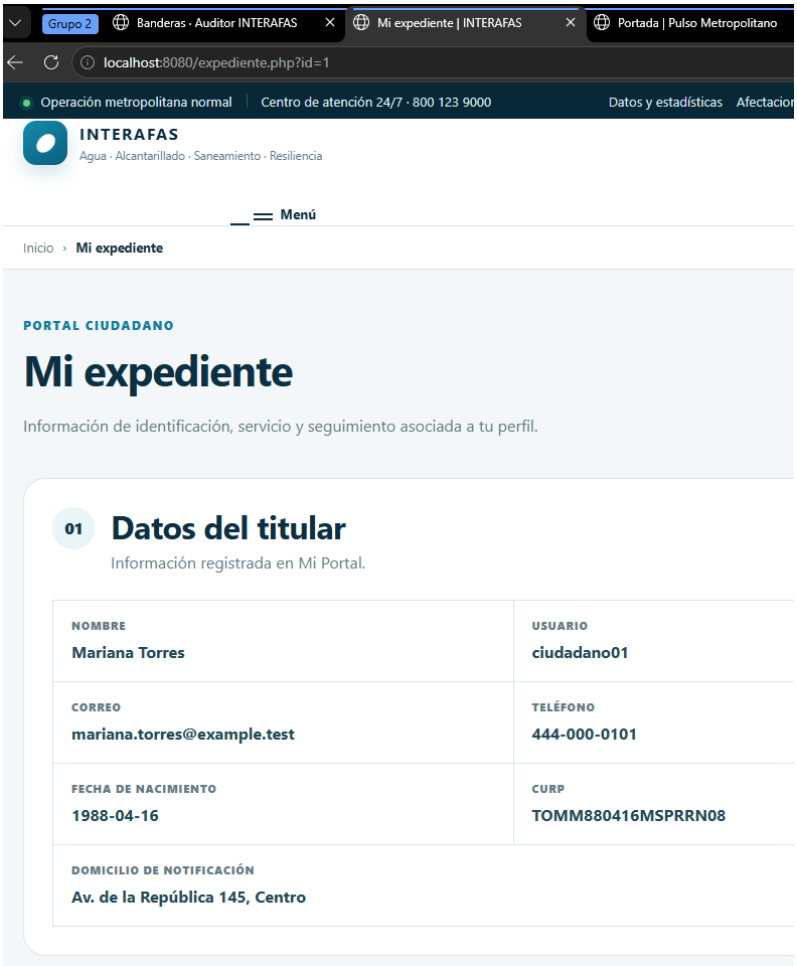


Figura 8. *Acceso exitoso a datos confidenciales inyectando el parámetro id=1 en la URL, evidenciando la falta de validación de identidad por parte del servidor.*

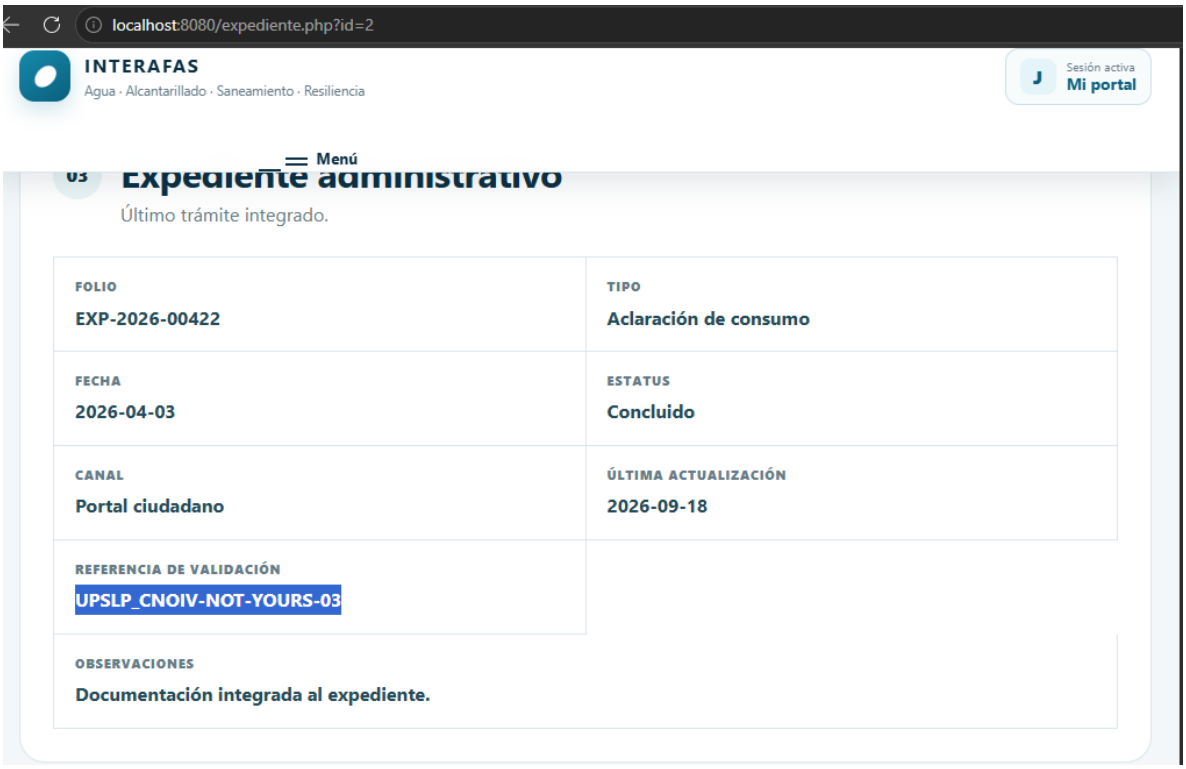


Figura 9. *Explotación de la vulnerabilidad IDOR mediante la iteración del parámetro secuencial a id=2, exponiendo información personal de terceros y revelando la bandera de validación UPSLP_CNOIV-NOT-YOURS-03.*

4.3.4 Impacto y riesgo

La explotación de esta vulnerabilidad tiene un impacto destructivo y directo sobre la Confidencialidad de la información gestionada por INTERAFAS. Al tratarse de un portal ciudadano, la falla expone de manera indiscriminada Datos Personales Identificables (PII) de los usuarios, incluyendo nombres completos, domicilios de notificación, claves únicas (CURP), fechas de nacimiento, correos electrónicos, números telefónicos y detalles sobre los trámites iniciados.

El riesgo operacional, legal y reputacional es crítico. Al carecer de controles de acceso y utilizar identificadores predecibles, un atacante externo podría automatizar un *script* simple para iterar secuencialmente sobre el parámetro (por ejemplo, desde id=1 hasta id=100000) y llevar a cabo una exfiltración masiva de la base de datos completa de ciudadanos (*data scraping*). Esta fuga de datos no solo viola el derecho a la privacidad de los individuos y normativas legales, sino que socava completamente la credibilidad de la infraestructura y expone a los ciudadanos a fraudes y robo de identidad.

4.3.5 Mitigaciones

Para remediar la causa raíz de este hallazgo, se recomiendan las siguientes medidas correctivas:

1. **Control de Autorización a Nivel de Objeto:** Modificar la lógica de *backend* en expediente.php para que el sistema valide que el ID del registro solicitado pertenece de forma explícita al ID del usuario actualmente autenticado en la sesión activa antes de devolver cualquier información.
2. **Eliminar Referencias Directas Predecibles:** Sustituir los identificadores numéricos y secuenciales en las URLs (como id=1 o id=2) por Identificadores Únicos Universales (UUID) o *hashes* no predecibles. Esto mitiga de inmediato los ataques de adivinación o iteración masiva.
3. **Manejo Estricto de Sesiones:** Si un visitante intenta consultar un expediente sin contar con un *token* de sesión válido, la aplicación debe rechazar la solicitud y redirigir inmediatamente a la página de inicio de sesión (HTTP 401 o 403), independientemente de los parámetros enviados.

4.4 Hallazgo 04 (VULN-04): Exposición de Código Fuente mediante Archivos de Respaldo Residuales (Reto 4)

4.4.1 Descripción técnica y causa raíz

Durante la revisión de los artefactos expuestos en la fase de reconocimiento (específicamente en las notas de despliegue release.info), se descubrió una advertencia sobre la existencia de archivos de respaldo generados antes de la sincronización en producción. A partir de esta pista, se identificó una vulnerabilidad de exposición de información sensible debida a la presencia de ficheros residuales. Al intentar acceder a páginas existentes añadiendo extensiones de respaldo comunes (como .bak), el servidor web devolvió el código fuente de la página administrativa en texto plano en lugar de ejecutarlo.

La causa raíz de esta vulnerabilidad se divide en dos deficiencias concurrentes de configuración (OWASP - *Security Misconfiguration*). Por un lado, existe un error crítico en los procesos operacionales de despliegue, permitiendo que copias de seguridad de los scripts permanezcan en el directorio público (DocumentRoot). Por otro lado, el servidor web Apache carece de un control de denegación por defecto para extensiones no reconocidas; al no estar configurado para interpretar la extensión .bak a través del motor de PHP, el servidor procesa la solicitud enviando el archivo completo como texto plano, revelando la lógica interna del lado del servidor.

4.4.2 Procedimiento de comprobación

El vector de ataque se validó empleando una técnica de descubrimiento de archivos iterativa manual (*fuzzing* lógico), siguiendo estos pasos:

1. Se analizaron las notas del desarrollador en el archivo release.info descubierto previamente, donde se indicaba textualmente: *"Pendiente: eliminar el archivo de respaldo generado antes del rsync final"* y *"Revisar que no queden artefactos .bak o .old bajo el document root"*.
2. En lugar de utilizar herramientas automatizadas de fuerza bruta de directorios (como *Gobuster* o *DirBuster*), se optó por un enfoque dirigido, añadiendo manualmente las extensiones .old y .bak a los nombres de las páginas principales detectadas en el portal.
3. Al probar la ruta correspondiente al módulo de transparencia, introduciendo la URL http://localhost:8080/transparencia.php.bak, el servidor no arrojó un error 404 ni ejecutó el código PHP.
4. En su lugar, el servidor descargó (o mostró en pantalla) el archivo en texto plano. Al inspeccionar el contenido del código fuente, se localizaron comentarios de los desarrolladores marcados como *TODO* y una variable PHP declarada como $releaseValidation, la cual contenía en claro la bandera del reto.

4.4.3 Evidencia técnica

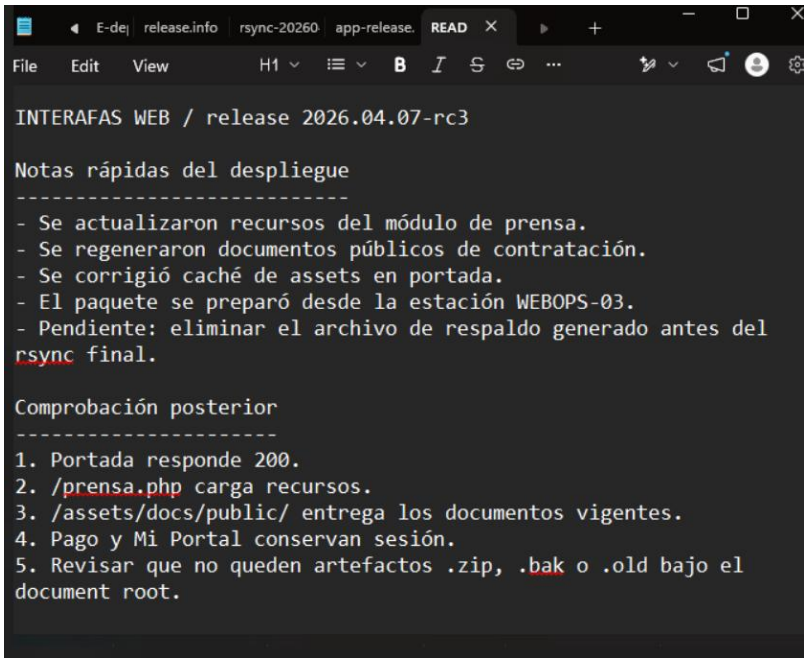


Figura 10. *Inspección del archivo de notas de despliegue (release.info) que expone la omisión procedimental de eliminar los artefactos de respaldo del servidor.*

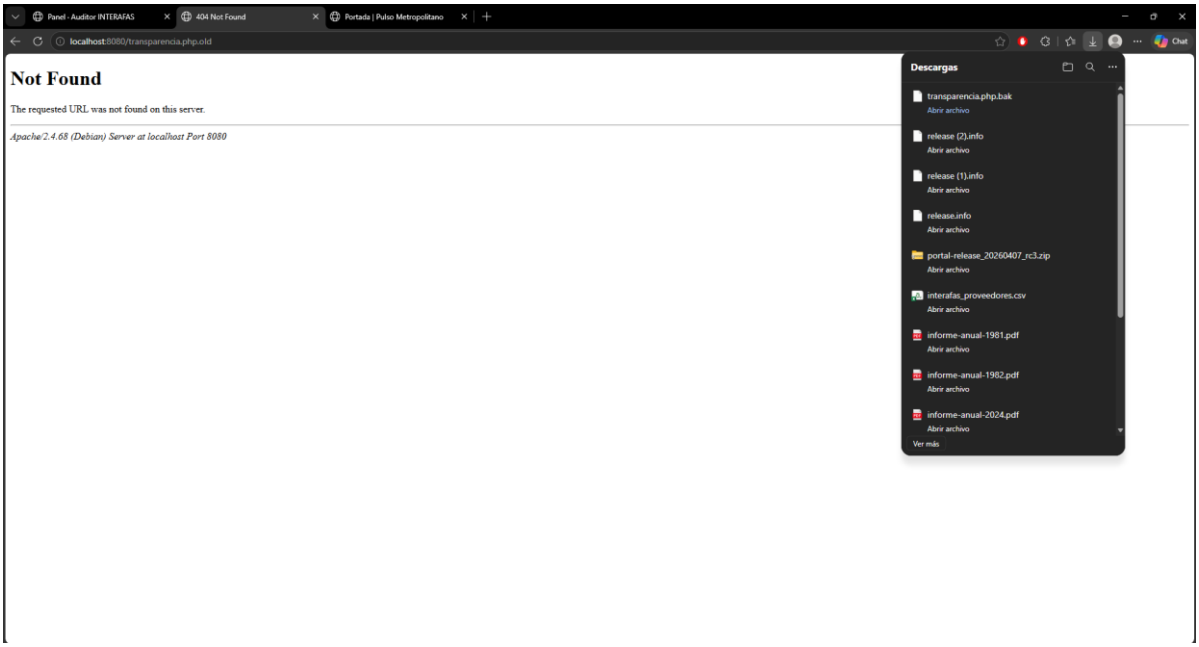


Figura 11. *Exposición del código fuente PHP descargado en texto plano a través del archivo residual transparencia.php.bak, revelando comentarios internos y la bandera de validación del reto.*

4.4.4 Impacto y riesgo

La explotación de archivos residuales representa una brecha significativa a la Confidencialidad del sistema, ya que anula completamente la protección que ofrece la ejecución del lado del servidor. Exponer el código fuente original de los aplicativos en PHP permite a un atacante auditar la lógica del negocio mediante tácticas de "caja blanca" (*White-box testing*). En la práctica, estos archivos frecuentemente contienen configuraciones codificadas en duro (*hardcoded*), como credenciales de bases de datos, claves de cifrado, direcciones IP de servidores internos y comentarios de desarrollo que delatan el funcionamiento exacto de los mecanismos de seguridad de la aplicación.

El riesgo operacional es alto porque facilita drásticamente la creación de vectores de ataque más sofisticados. Al conocer la estructura del código, las validaciones precisas que hace el sistema y las vulnerabilidades subyacentes, un adversario puede diseñar inyecciones SQL, evadir filtros de seguridad o realizar escaladas de privilegios con precisión quirúrgica, sin generar el ruido típico que produce un escaneo automatizado a ciegas.

4.4.5 Mitigaciones

Para solucionar definitivamente este hallazgo, se deben aplicar las siguientes mitigaciones técnicas y operativas:

1. **Restricción de extensiones a nivel servidor:** Modificar la configuración de Apache (por ejemplo, a través de directivas en httpd.conf o apache2.conf) para denegar explícitamente el acceso web a cualquier archivo que posea extensiones de respaldo (como *.bak, *.old, *.swp, *.zip, *.tar.gz, *.env), retornando un código HTTP 403 Forbidden.
2. **Implementación de despliegues automatizados seguros:** Reemplazar el uso de copias manuales o secuencias rsync inseguras por canalizaciones (*pipelines*) de despliegue continuo (CI/CD) que garanticen que solo los archivos ejecutables aprobados en el control de versiones (Git) sean transferidos al entorno de producción, realizando un barrido de limpieza de artefactos en el proceso.
3. **Auditoría de integridad de archivos (FIM):** Instalar sistemas de monitoreo de integridad en el servidor (como OSSEC o Wazuh) que alerten al equipo de seguridad en caso de que se creen o modifiquen archivos inesperados dentro del directorio público de la aplicación web.

4.5 Hallazgo 05 (VULN-05): Exposición de Información por Manejo Inadecuado de Excepciones (Reto 5)

4.5.1 Descripción técnica y causa raíz

Durante la auditoría de los flujos de interacción del usuario, se identificó una vulnerabilidad de exposición de información sensible desencadenada por un manejo inadecuado de excepciones en el servidor. Al interactuar con el módulo de exportación de datos públicos, se descubrió que el aplicativo procesa el formato de descarga a través de un parámetro en la URL. Al manipular este parámetro insertando caracteres especiales no esperados, la lógica de *backend* falló de manera abrupta, revelando una pantalla de diagnóstico de errores detallada (modo *debug*) en lugar de un mensaje genérico de error.

La causa raíz de este hallazgo corresponde a una configuración de seguridad deficiente (OWASP - *Security Misconfiguration*). La aplicación se encuentra desplegada en un entorno de producción (production), pero mantiene habilitadas las respuestas de depuración (DEBUG RESPONSE ENABLED). Adicionalmente, el código carece de un bloque de control global de excepciones (como *try-catch*) y de validación de entrada (lista blanca) para el parámetro afectado. Al recibir un dato no contemplado en su estructura lógica (un bloque match en PHP), el sistema arroja un UnhandledMatchError, derramando toda la información del entorno directamente al cliente.

4.5.2 Procedimiento de comprobación

El vector de ataque se validó provocando intencionalmente un error de sintaxis en la lógica de la aplicación mediante los siguientes pasos:

1. Se analizó el comportamiento de la plataforma, identificando los módulos que afectaban la descarga de archivos (específicamente en las secciones de proveedores y contrataciones donde se ofrecen exportaciones en CSV, XLSX y JSON).
2. Se interceptó y analizó la URL de descarga, identificando que el formato se controlaba mediante el parámetro GET tipo (por ejemplo, exportar.php?tipo=csv).
3. Se manipuló la URL en la barra de direcciones, inyectando una cadena de caracteres especiales (@#$%^&*^#$%^<>?) en el parámetro tipo para forzar un comportamiento anómalo.
4. Al enviar la petición, el servidor no pudo clasificar el formato y generó una excepción no controlada (Unhandled export exception).
5. La respuesta del servidor mostró una página de diagnóstico detallada que expuso la ruta del archivo fuente (/var/www/html/exportar.php), detalles del servicio de base de datos (db:3306 / schema interafas) y la bandera de validación del reto.

4.5.3 Evidencia técnica

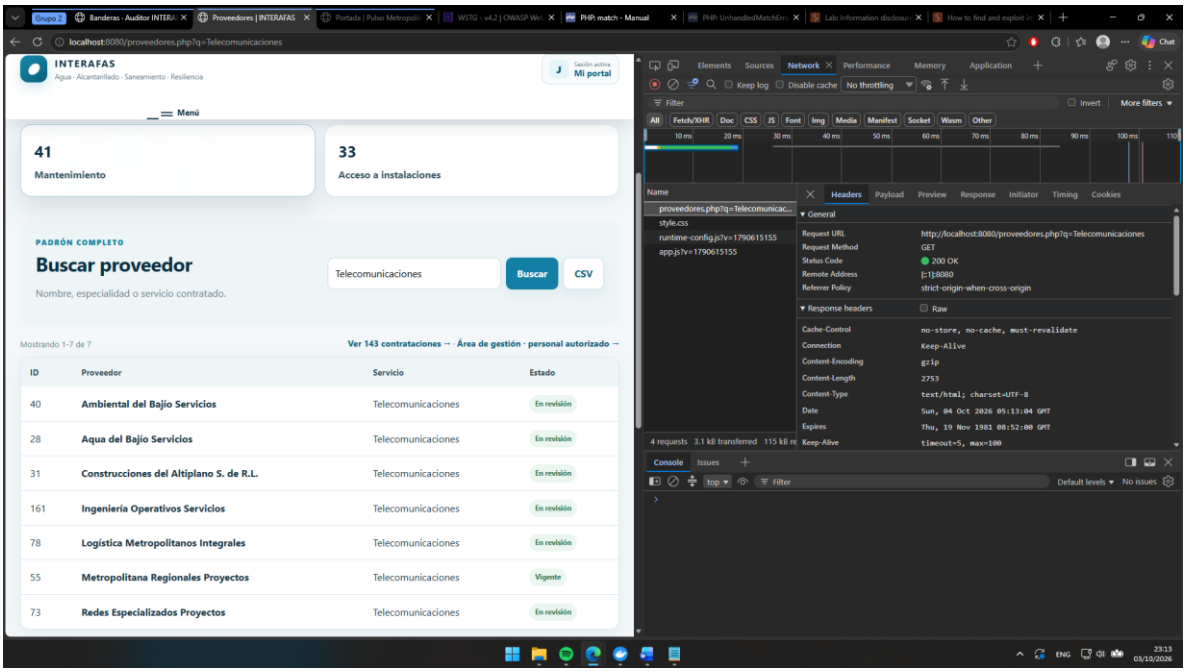


Figura 12. *Identificación de los puntos de entrada para la exportación de datos en los módulos del portal, los cuales utilizan parámetros GET para definir el formato del archivo.*

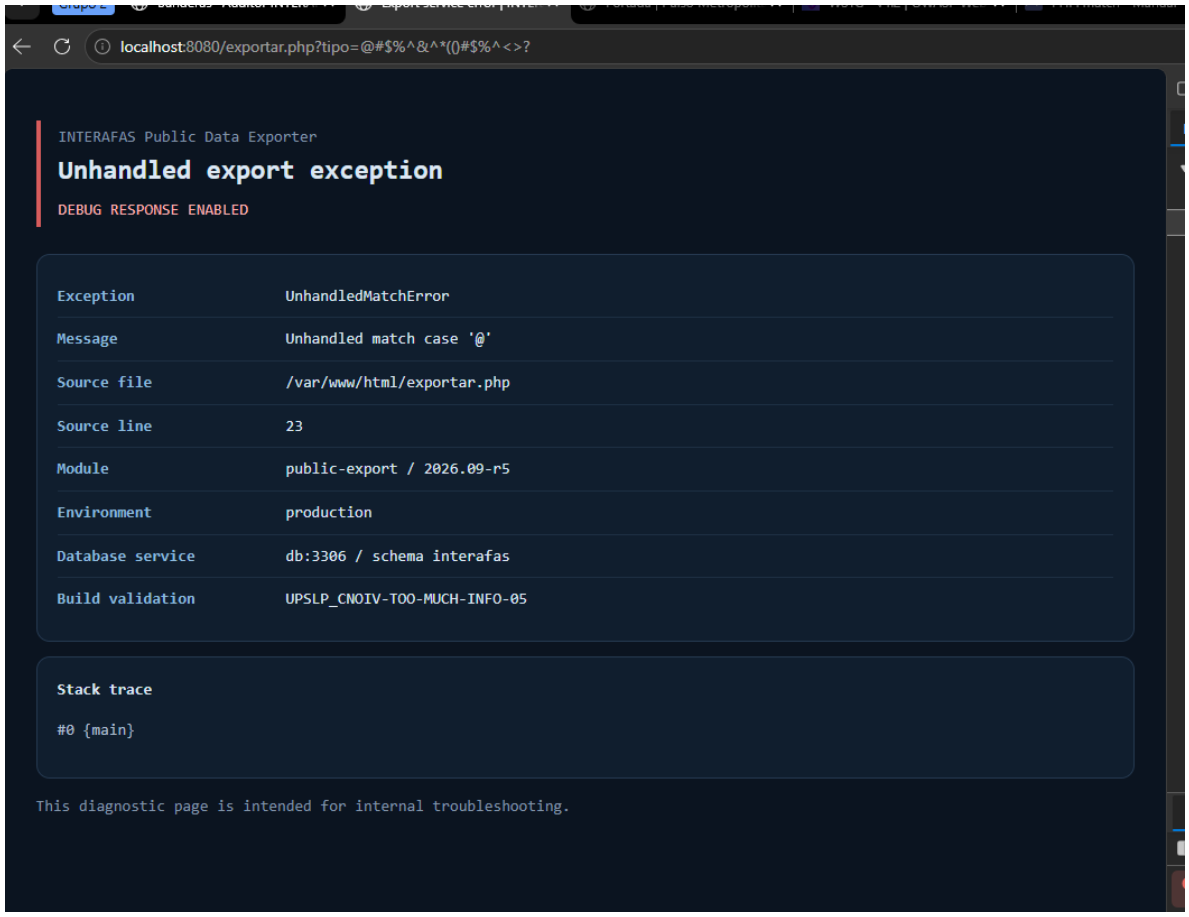


Figura 13. *Explotación exitosa mediante la alteración del parámetro "tipo" con caracteres especiales, provocando la caída de la aplicación y la revelación de la pantalla de depuración con metadatos críticos del servidor.*

4.5.4 Impacto y riesgo

La fuga de información mediante mensajes de error detallados vulnera directamente la Confidencialidad de la arquitectura del servidor. Aunque esta vulnerabilidad no permite por sí sola la ejecución de código o la alteración de datos, proporciona a un atacante un mapa técnico invaluable del entorno de alojamiento. Conocer las rutas absolutas del sistema de archivos, los nombres exactos de los esquemas de bases de datos y los puertos internos de conexión elimina el trabajo de adivinación (*guessing*) necesario para estructurar ataques avanzados.

El riesgo operacional de mantener el modo *debug* activado en producción es que actúa como un facilitador de compromisos críticos. Si posteriormente se descubre una vulnerabilidad de inyección SQL o de inclusión de archivos locales (LFI) en otro módulo del portal, el atacante utilizará las rutas y nombres de esquemas filtrados en esta pantalla de error para dirigir su *exploit* con total exactitud, comprometiendo rápidamente la infraestructura de INTERAFAS.

4.5.5 Mitigaciones

Para solucionar definitivamente esta vulnerabilidad, se recomiendan las siguientes acciones técnicas:

1. **Desactivar la visualización de errores:** Modificar la configuración del servidor web y del motor de PHP (php.ini) para asegurar que la directiva display_errors esté establecida en Off en el entorno de producción.
2. **Manejo global de excepciones:** Implementar manejadores de excepciones personalizados en el código (*Custom Exception Handlers*). Si ocurre un error no controlado (como un UnhandledMatchError), el sistema debe atraparlo, registrar los detalles técnicos internamente en un archivo de bitácora (*log*) seguro y devolver al usuario únicamente una vista genérica de "Error HTTP 500 - Error Interno del Servidor".
3. **Validación de entrada (*Allowlisting*):** Establecer una validación estricta sobre el parámetro tipo. El código debe verificar que el valor recibido coincida exactamente con una lista blanca predefinida (ej. ['csv', 'xlsx', 'json']). Si la entrada no coincide, la solicitud debe ser rechazada inmediatamente (HTTP 400 Bad Request) antes de ser procesada por la lógica de negocio.

4.6 Hallazgo 06 (VULN-06): Escalada de Privilegios Vertical por Confianza en Datos del Cliente (Reto 6)

4.6.1 Descripción técnica y causa raíz

Durante la auditoría de los controles de autorización de la plataforma, se descubrió una vulnerabilidad crítica de escalada de privilegios vertical. Al intentar acceder al módulo administrativo de gestión de proveedores, el sistema bloqueó el acceso mostrando un mensaje de "Acceso no autorizado". Sin embargo, el mecanismo de protección implementado por el servidor era inherentemente inseguro, ya que utilizaba una *cookie* en texto plano (portal_role) almacenada en el navegador del usuario para determinar su nivel de privilegios. Al manipular el valor de esta *cookie*, fue posible engañar al servidor para que otorgara acceso total a funciones reservadas para administradores.

La causa raíz de esta vulnerabilidad es una violación directa a las prácticas de diseño seguro, categorizada por OWASP como *Broken Access Control*. El error arquitectónico consiste en depositar la confianza de la autorización en el lado del cliente (el navegador) en lugar de hacerlo en el lado del servidor. El *backend* lee el valor de la *cookie* de rol modificable y lo acepta como una aserción de identidad válida, omitiendo cruzar esta información con una base de datos o un registro de sesión seguro vinculado de manera inmutable al usuario.

4.6.2 Procedimiento de comprobación

El vector de ataque se validó de manera exitosa mediante la siguiente secuencia metodológica manual:

1. Se inició sesión en el portal con una cuenta de ciudadano estándar y se navegó al padrón público de proveedores (/proveedores.php).
2. Se intentó acceder a la ruta administrativa restringida (/gestion-proveedores.php), lo que provocó una denegación de acceso (HTTP 403) con el mensaje de que la sesión carecía de autorización.
3. Utilizando las herramientas de desarrollador del navegador (F12) en la pestaña *Network*, se analizaron las cabeceras de la petición denegada, observando que el servidor exigía el rol procurement_admin (X-Required-Role: procurement_admin) y que el cliente enviaba su rol actual mediante la *cookie* portal_role=citizen.
4. En la pestaña de almacenamiento (*Storage* / *Application*), se editó manualmente el valor de la *cookie* portal_role, reemplazando la palabra citizen por procurement_admin.
5. Se refrescó la página /gestion-proveedores.php. El servidor confió en el dato manipulado, concediendo acceso completo al módulo de gestión y revelando la bandera de validación en una notificación del sistema.

4.6.3 Evidencia técnica

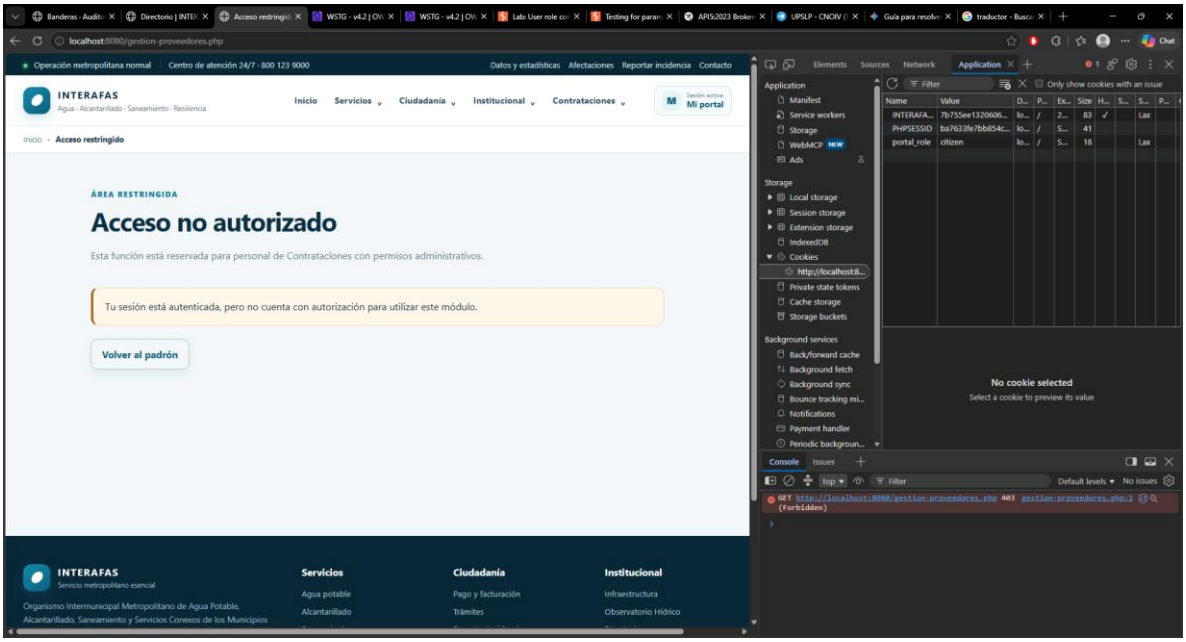


Figura 14. *Denegación de acceso inicial al módulo de gestión de proveedores, confirmando que la sesión estándar no posee los privilegios requeridos por el sistema.*

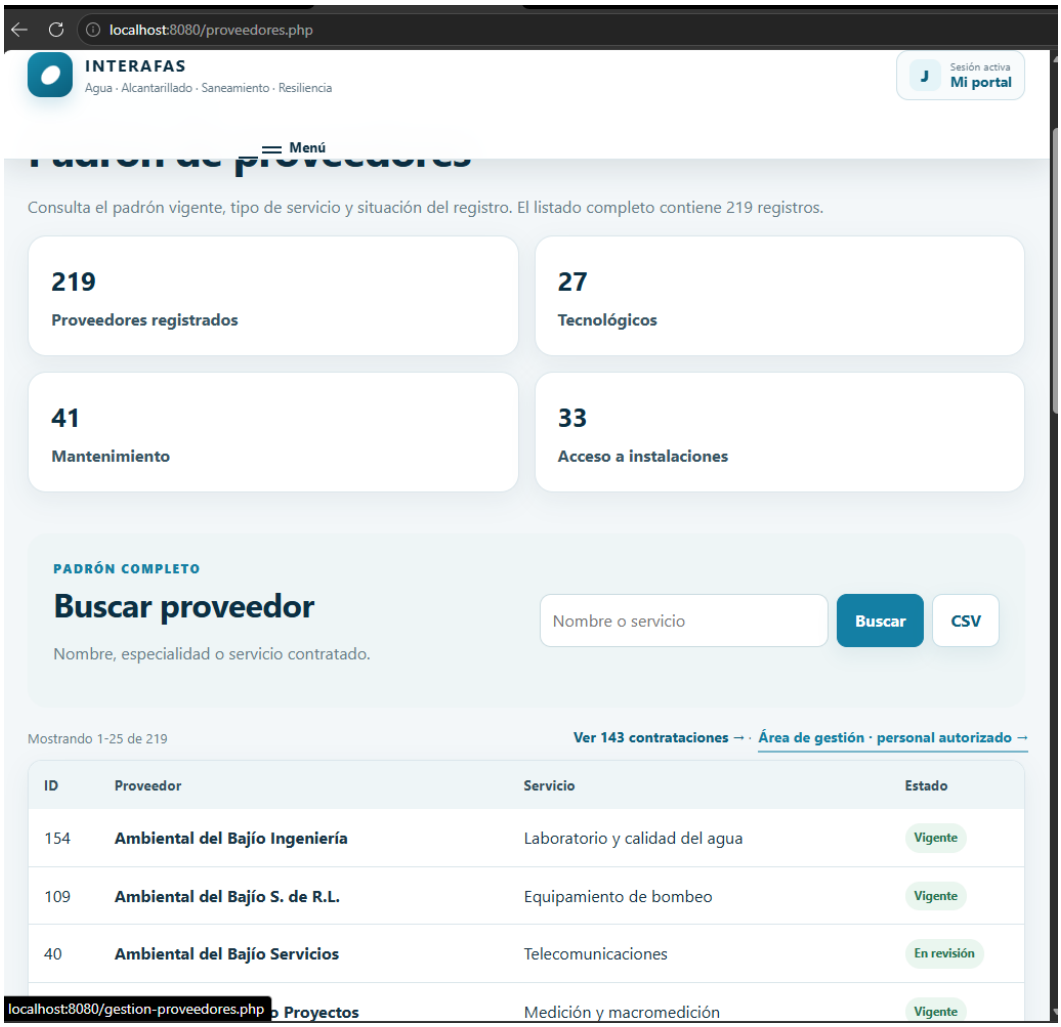


Figura 15. *Navegación desde el directorio público hacia la ruta administrativa, paso previo a la intercepción y análisis de las cabeceras de autenticación.*



Figura 16. *Explotación exitosa de escalada de privilegios al alterar el valor de la cookie "portal_role". El servidor otorga acceso a las funciones administrativas y expone la bandera del reto.*

4.6.4 Impacto y riesgo

La escalada de privilegios vertical representa un quiebre total del modelo de seguridad de la aplicación, impactando gravemente la Integridad y la Confidencialidad de la organización. Al lograr acceso administrativo, un atacante o un usuario malintencionado supera las barreras lógicas diseñadas para proteger procesos operativos críticos. En este contexto específico, el atacante obtiene la capacidad de gestionar el padrón de proveedores, lo que podría derivar en la aprobación de contratos fraudulentos, la alteración de estatus de proveedores sancionados o el borrado de auditorías internas.

A nivel de riesgo sistémico, confiar en el cliente para el control de acceso es una de las fallas arquitectónicas más peligrosas. Demuestra que el control de acceso basado en roles (RBAC) es superficial. Esta vulnerabilidad permite a cualquier usuario registrado, sin importar sus conocimientos técnicos, elevar sus privilegios utilizando herramientas básicas integradas en cualquier navegador web, convirtiendo un riesgo teórico en una amenaza de explotación trivial e inmediata.

4.6.5 Mitigaciones

Para solucionar definitivamente esta vulnerabilidad, se recomiendan las siguientes medidas correctivas:

1. **Autorización del Lado del Servidor:** Eliminar inmediatamente el uso de *cookies*, parámetros GET/POST o campos ocultos en HTML para definir o transportar el nivel de privilegios del usuario. El rol debe ser consultado exclusivamente desde la base de datos o desde un registro de sesión seguro alojado en la memoria del servidor y referenciado únicamente por un identificador de sesión opaco (PHPSESSID).
2. **Implementación Estricta de RBAC:** Cada *endpoint* o archivo PHP restringido (como gestion-proveedores.php) debe implementar una función de validación que consulte al servidor si el identificador de la sesión actual corresponde a un usuario con los permisos administrativos necesarios antes de renderizar cualquier contenido.
3. **Uso de Tokens Firmados (Mitigación alternativa para APIs):** En caso de utilizar arquitecturas desacopladas o sin estado (*stateless*), emplear JSON Web Tokens (JWT) correctamente implementados, donde el rol esté contenido dentro del *payload* criptográficamente firmado por el servidor, haciendo imposible su manipulación por parte del cliente sin invalidar la firma.

4.7 Hallazgo 07 (VULN-07): Inyección SQL Ciega (Blind SQLi) Basada en Booleanos (Reto 7)

4.7.1 Descripción técnica y causa raíz

Durante la evaluación de los módulos de consulta pública, se identificó una vulnerabilidad crítica de Inyección SQL (OWASP - *Injection*) en el formulario de validación de referencias de procedimientos de contratación (validar-referencia.php). Específicamente, el parámetro GET ref resultó ser vulnerable a una Inyección SQL Ciega basada en booleanos (*Boolean-based blind SQLi*). A diferencia de una inyección clásica donde los datos extraídos se reflejan en la pantalla, en este escenario la aplicación no mostraba los errores de la base de datos, pero su comportamiento (mostrar el mensaje "Referencia localizada" frente a no mostrarlo) permitía inferir si una consulta inyectada era verdadera o falsa.

La causa raíz de esta vulnerabilidad reside en la ausencia de parametrización en las consultas a la base de datos. El código del lado del servidor (desarrollado en PHP) concatena directamente la entrada proporcionada por el usuario (el valor de ref) dentro de la sentencia SQL que se ejecuta en el motor MariaDB/MySQL del *backend*. Al no emplear sentencias preparadas (*Prepared Statements*) ni sanitizar la entrada de caracteres de control SQL (como comillas simples o palabras reservadas como AND), el servidor interpreta la carga útil maliciosa como instrucciones ejecutables legítimas.

4.7.2 Procedimiento de comprobación

El vector de ataque se validó combinando el análisis manual de la aplicación con la herramienta de explotación automatizada sqlmap, siguiendo esta secuencia metodológica:

1. Se analizó el formulario de validación pública, notando que al enviar un formato válido (como LP-INT-000-2000) la URL transmitía el dato mediante el parámetro ?ref= y la interfaz devolvía un mensaje de confirmación lógico.

2. Se configuró y ejecutó la herramienta sqlmap desde una terminal de PowerShell, dirigiéndola hacia el *endpoint* vulnerable y especificando el parámetro ref como objetivo, indicando a la herramienta que buscara la cadena "Referencia localizada" como indicador de una respuesta verdadera (--string="Referencia localizada").

3. La herramienta confirmó que el parámetro era inyectable mediante la técnica *AND boolean-based blind WHERE or HAVING clause*.

4. Se procedió a enumerar las bases de datos del servidor, identificando exitosamente el esquema principal denominado interafas.

5. Posteriormente, se enumeraron las tablas de dicha base de datos, localizando una tabla de interés llamada portal_validation_meta y extrayendo sus columnas (context_key, id, verification_token).

6. Finalmente, se ejecutó un comando de volcado (*dump*) específicamente sobre la columna verification_token, logrando extraer en texto claro la bandera de validación del reto.

4.7.3 Evidencia técnica

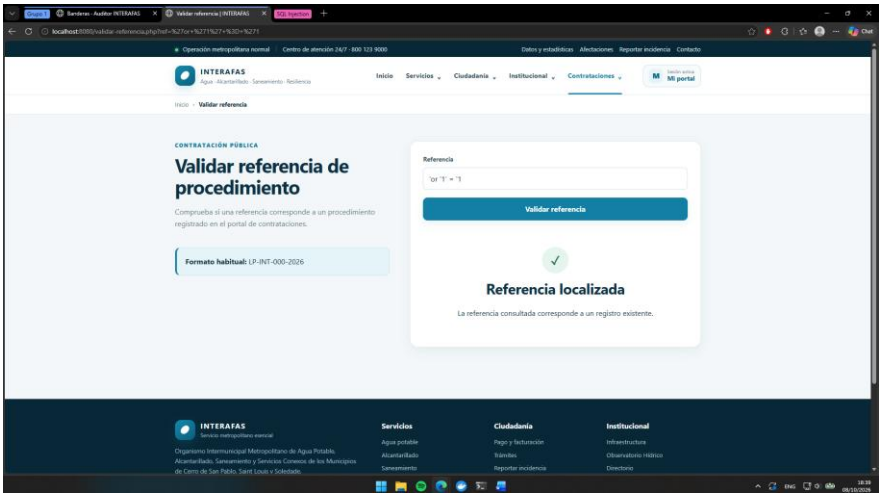


Figura 17. *Interacción con el módulo de validación de referencias, identificando el parámetro GET vulnerable y el mensaje de confirmación que sirve como base para la inyección ciega.*



Figura 18. *Ejecución de la herramienta automatizada sqlmap confirmando que el parámetro "ref" es inyectable mediante la técnica de inyección ciega basada en booleanos.*



Figura 19. *Fase de enumeración y extracción de esquemas del backend, listando las tablas internas de INTERAFAS y descubriendo la estructura de columnas del registro de validación.*

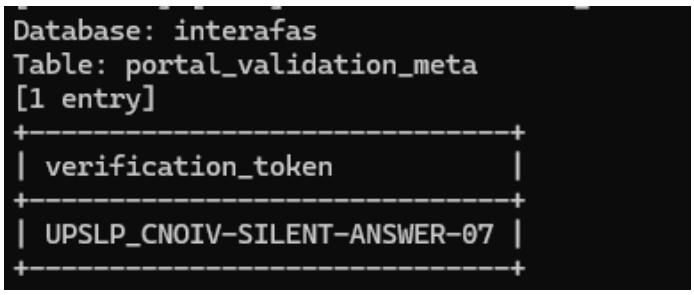


Figura 20. *Exfiltración exitosa (Data Dump) del contenido de la base de datos a través de la inyección ciega, revelando la bandera oculta dentro de la columna verification_token.*

4.7.4 Impacto y riesgo

La materialización de esta Inyección SQL representa una vulnerabilidad de impacto devastador para la Confidencialidad y la Integridad de la plataforma. Aunque la inyección es ciega y requiere que el atacante infiera los datos carácter por carácter basándose en respuestas lógicas verdaderas o falsas, las herramientas automatizadas modernas permiten extraer el contenido íntegro de la base de datos en cuestión de minutos. Esto significa que un atacante puede exfiltrar todas las tablas del sistema, incluyendo credenciales administrativas, registros de infraestructura crítica ciudadana y configuraciones operativas internas.

El riesgo operacional es considerado crítico, ya que la base de datos comprometida es el núcleo de información de la aplicación web. Dependiendo de los privilegios asignados al usuario de conexión a la base de datos (por ejemplo, si el servicio corre como root o posee permisos de FILE), esta inyección SQL podría escalar rápidamente, permitiendo al atacante leer archivos del sistema operativo subyacente (como el archivo de contraseñas /etc/passwd) o, en el peor de los casos, escribir un archivo de *shell* web (Web Shell) en el servidor, logrando así una ejecución remota de comandos (RCE) y el control total de la máquina.

4.7.5 Mitigaciones

Para solucionar definitivamente la causa raíz de este hallazgo, se deben implementar de forma obligatoria las siguientes prácticas de codificación segura:

1. **Uso de Consultas Parametrizadas (Prepared Statements):** Es mandatorio reescribir la lógica de acceso a datos utilizando PDO (*PHP Data Objects*) o MySQLi con sentencias preparadas. Esta técnica separa de forma estricta la estructura del código SQL de los datos proporcionados por el usuario, imposibilitando que la entrada sea interpretada como comandos ejecutables.

2. **Validación y Saneamiento de Entrada (Allowlisting):** Implementar expresiones regulares (Regex) estrictas del lado del servidor que aseguren que el parámetro ref cumple exactamente con el formato esperado (por ejemplo, ^LP-INT-\d{3}-\d{4}$). Si la entrada contiene caracteres anómalos, la petición debe ser descartada antes de interactuar con la base de datos.

3. **Principio de Mínimo Privilegio en Base de Datos:** Configurar la conexión entre la aplicación web y el motor de base de datos utilizando un usuario con permisos altamente restringidos. Este usuario debe tener permisos exclusivos de lectura (SELECT) y escritura (INSERT, UPDATE) únicamente sobre las tablas estrictamente necesarias, revocando cualquier privilegio administrativo o de acceso a nivel de sistema.

4.8 Hallazgo 08 (VULN-08): Inyección de Contenido y Manipulación de Metadatos en Módulo de Reportes (Reto 8)

4.8.1 Descripción técnica y causa raíz

Durante la auditoría del módulo de atención ciudadana y seguimiento de incidencias, se identificó una vulnerabilidad relacionada con el procesamiento y renderizado de entradas de usuario (clasificada dentro de OWASP como *Injection* / *Cross-Site Scripting* o deficiencias en el manejo de contenido almacenado). Al generar un reporte ciudadano, los datos introducidos en la descripción del problema se almacenaban en el sistema y se reflejaban posteriormente en la vista de comprobante y seguimiento (seguimiento-reporte.php). La aplicación ejecutaba llamadas asíncronas de validación (validate-stored-content.php) que analizaban el DOM o los metadatos de la página, permitiendo que la inserción de contenido estructurado interactuara con los scripts de verificación del lado del cliente.

La causa raíz de esta vulnerabilidad radica en la ausencia de un saneamiento y validación estricta de los datos ingresados por el usuario antes de ser almacenados y devueltos por el servidor (*insufficient input sanitization*), así como en la falta de codificación de salida (*output encoding*). Al permitir que caracteres especiales o etiquetas HTML/JavaScript formen parte del flujo de datos del reporte, la lógica de validación interna del sistema procesa e interpreta dicho contenido, exponiendo información reservada o tokens de control que deberían permanecer inaccesibles para usuarios sin privilegios.

4.8.2 Procedimiento de comprobación

El vector de análisis y validación se ejecutó siguiendo esta secuencia metodológica:

1. Se ingresó a la sección de reportes ciudadanos de la plataforma y se creó un nuevo reporte de incidencia rellenando los campos obligatorios de categoría, municipio y descripción.
2. Al finalizar el registro, el sistema generó un comprobante con un identificador único de seguimiento (por ejemplo, REP-20261010-1D826F) y redirigió a la vista de detalles.
3. Utilizando las herramientas de desarrollador del navegador (F12) en la pestaña *Network* y la consola (*Console*), se inspeccionaron las peticiones de tipo *fetch* asociadas a la página, específicamente el recurso /validate-stored-content.php.
4. Se observó que el *script* de validación analizaba los elementos del DOM y las etiquetas *meta* incrustadas en la respuesta del servidor.
5. Al revisar el resultado de la traza en la consola y la pestaña *Response* de la petición de validación, el sistema extrajo y mostró con éxito un token de control y validación vinculado al almacenamiento del reporte.

4.8.3 Evidencia técnica

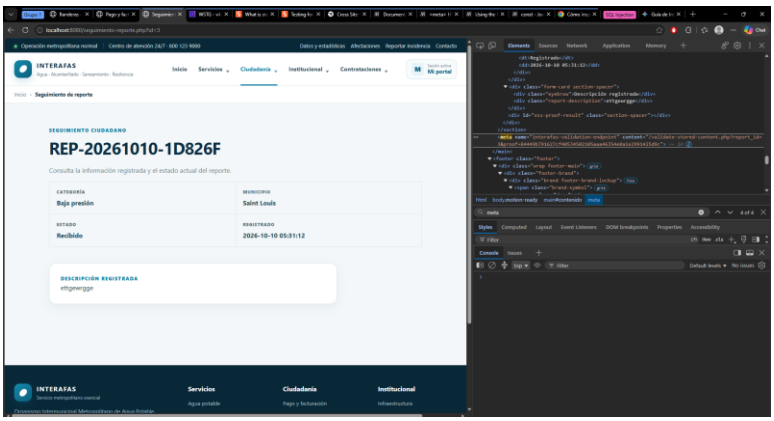


Figura 21. *Vista del comprobante de seguimiento de reporte ciudadano generado en el sistema, mostrando el identificador único de la incidencia.*

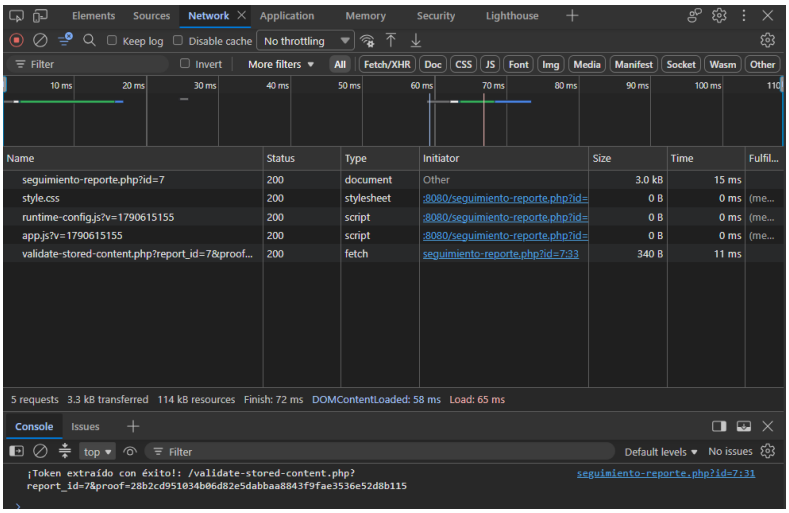


Figura 22. *Inspección de la consola y el panel de red del navegador, evidenciando la ejecución de scripts de validación que interactúan con el contenido almacenado del reporte.*

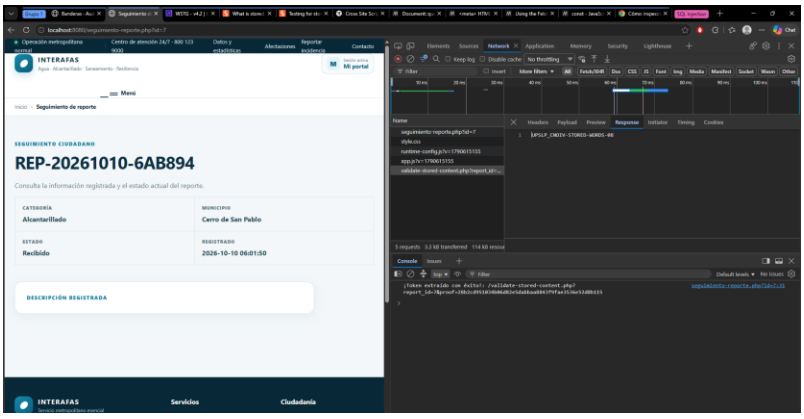


Figura 23. *Validación del flujo de respuesta del servidor en el módulo de seguimiento, donde la lectura de metadatos expone el comportamiento anómalo en la manipulación de contenidos.*

4.8.4 Impacto y riesgo

La presencia de vulnerabilidades de inyección en formularios de cara al público representa un riesgo moderado a alto sobre la Confidencialidad y la Integridad de la plataforma web. Si un atacante logra almacenar contenido malicioso (como scripts de JavaScript arbitrarios) en los reportes que visualizan los operadores internos, se materializaría un ataque de XSS Almacenado (*Stored Cross-Site Scripting*). Esto permitiría el secuestro de sesiones de administradores, la manipulación de la interfaz visual del panel de control o la exfiltración de información confidencial de otros usuarios.

A nivel operativo, este tipo de fallos demuestra una deficiencia en el ciclo de validación de entradas de los módulos interactivos. En infraestructuras críticas, permitir que datos no confiables modifiquen el comportamiento del DOM o ejecuten rutinas de validación del lado del cliente abre la puerta a la suplantación de acciones y a la alteración de los registros de auditoría de incidentes, comprometiendo la trazabilidad oficial de los eventos reportados.

4.8.5 Mitigaciones

Para solucionar la causa raíz de este hallazgo, se deben implementar las siguientes medidas correctivas:

1. **Codificación de Salida Contextual (*Output Encoding*):** Asegurar que cualquier dato ingresado por el usuario y mostrado posteriormente en la interfaz web sea codificado de forma estricta según su contexto (HTML, JavaScript, atributos o URLs) antes de ser renderizado por el navegador, neutralizando la ejecución de etiquetas o código incrustado.
2. **Saneamiento y Validación Estricta (*Input Validation*):** Aplicar listas blancas de caracteres permitidos en los campos de texto de los reportes ciudadanos, rechazando de inmediato cualquier cadena que contenga símbolos especiales, etiquetas HTML o fragmentos de código ejecutable.
3. **Implementación de Cabeceras de Seguridad (CSP):** Configurar adecuadamente la política de seguridad de contenidos (*Content Security Policy* - CSP) en el servidor web para restringir la ejecución de scripts no autorizados y mitigar de manera preventiva el impacto de posibles inyecciones de código en el cliente.

4.9 Hallazgo 09 (VULN-09): Vulnerabilidad de Recorrido de Directorios (Path / Directory Traversal) en Descarga de Archivos (Reto 9)

4.9.1 Descripción técnica y causa raíz

Durante la auditoría del módulo de gestión documental y constancias ciudadanas, se identificó una vulnerabilidad crítica de seguridad clasificada como Recorrido de Directorios o *Path Traversal* (*Directory Traversal*), la cual se enmarca dentro de las fallas de control de acceso y validación de entradas de OWASP. Al interactuar con la funcionalidad de descarga de documentos del portal, la aplicación procesaba la solicitud utilizando un parámetro basado en el nombre del archivo (file). El servidor tomaba este parámetro de manera directa para localizar y servir el recurso desde el sistema de archivos del *backend* sin aplicar un saneamiento adecuado ni restringir el directorio base de búsqueda.

La causa raíz de esta vulnerabilidad recae en una deficiencia severa en la validación de la entrada del usuario y en el control de límites del sistema de archivos (*insufficient input validation* y *improper limitation of a pathname to a restricted directory*). El código en PHP del lado del servidor utilizaba el valor recibido en el parámetro GET sin neutralizar las secuencias de escape de directorios (como los caracteres ../ o ./), permitiendo que un atacante saliera de la raíz prevista (DocumentRoot) y navegara libremente a través de la jerarquía de directorios del sistema operativo subyacente.

4.9.2 Procedimiento de comprobación

El vector de ataque se validó de manera exitosa mediante la siguiente secuencia metodológica manual:

1. Se exploró la sección de constancias y documentos del portal ciudadano, identificando que los enlaces de descarga apuntaban a un script centralizado mediante la estructura descargar-documento.php?file=nombre_archivo.pdf.
2. Se interceptó y manipuló la URL en la barra de direcciones del navegador, sustituyendo el nombre del archivo legítimo por una secuencia de saltos de directorio combinados con una ruta absoluta del sistema operativo Linux.
3. Se introdujo la carga útil de prueba solicitando el archivo de contraseñas del sistema: http://localhost:8080/descargar-documento.php?file=./../../../../../etc/passwd.
4. El servidor procesó la ruta anómala sin rechazarla y devolvió exitosamente el contenido del archivo crítico del sistema, exponiendo las cuentas de usuario y los intérpretes de comandos configurados.
5. Posteriormente, se realizó una prueba adicional consultando el archivo de mensajes del sistema operativo (/etc/motd), donde se localizó incrustada la bandera de validación correspondiente al reto.

4.9.3 Evidencia técnica

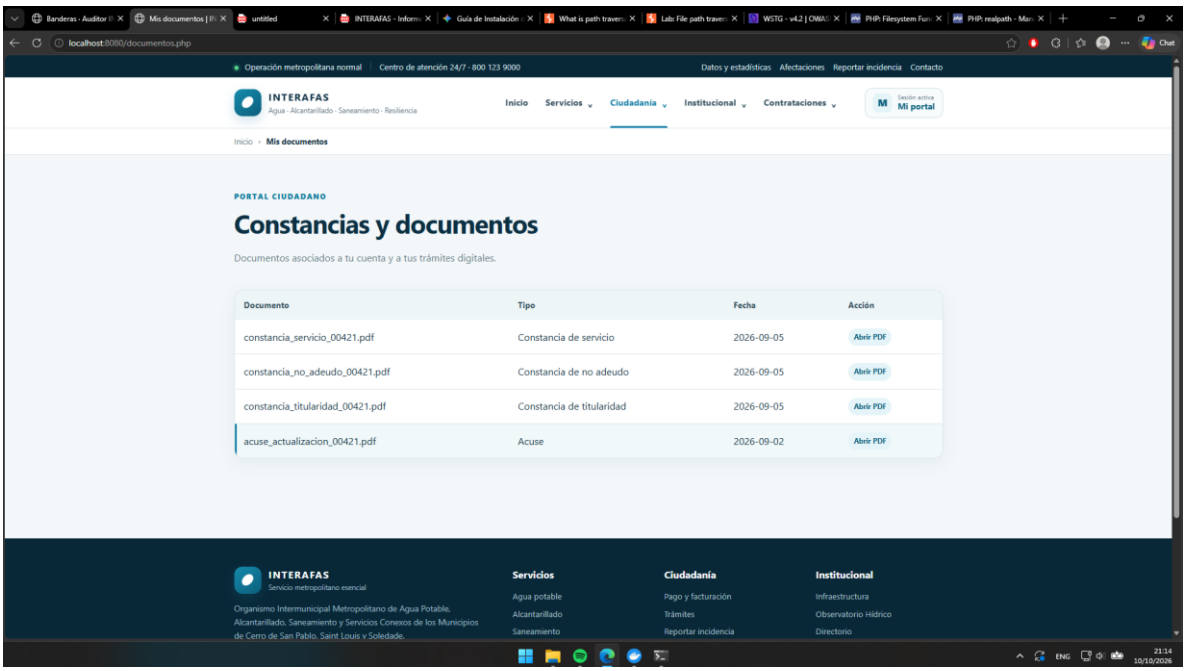


Figura 24. *Interfaz del portal ciudadano listando los documentos disponibles para descarga, identificando el script centralizado que procesa los archivos mediante el parámetro "file".*

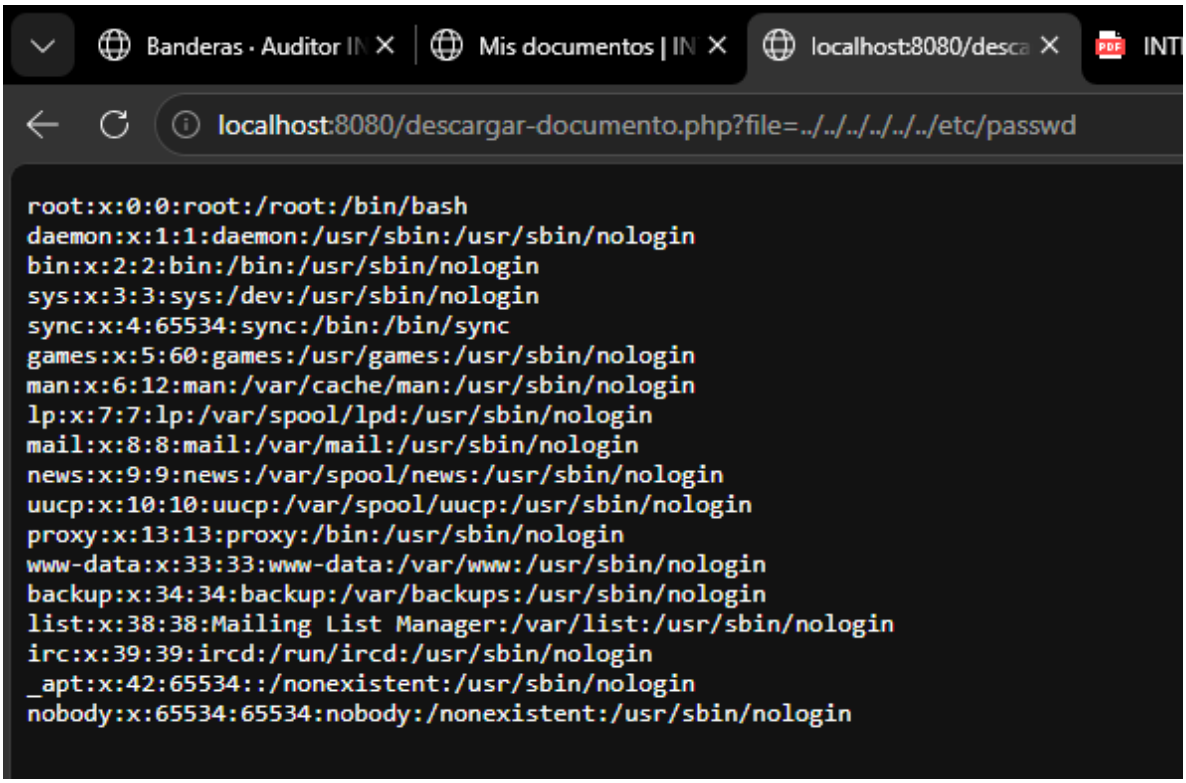


Figura 25. *Explotación exitosa de Path Traversal al inyectar secuencias de escape de directorios, logrando la lectura no autorizada del archivo sensible /etc/passwd del servidor host.*

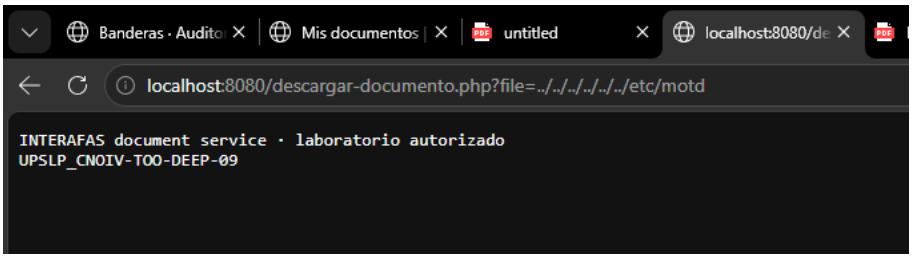


Figura 26. *Lectura arbitraria de ficheros del sistema operativo mediante el parámetro vulnerable, revelando el mensaje del día (/etc/motd) que contiene la bandera de validación del reto.*

4.9.4 Impacto y riesgo

La materialización de una vulnerabilidad de *Path Traversal* representa un impacto catastrófico sobre la Confidencialidad del servidor y de la infraestructura subyacente. Al permitir que un usuario externo lea archivos arbitrarios del sistema de archivos, el atacante no se limita a los datos de la aplicación web, sino que puede obtener acceso a archivos de configuración crítica del servidor web, código fuente con credenciales conectadas a bases de datos, llaves privadas SSH, registros de actividad y ficheros del sistema operativo como /etc/passwd.

El riesgo operativo e institucional es crítico. Si el proceso de la aplicación web cuenta con privilegios elevados o si el servidor está insuficientemente segmentado, esta vulnerabilidad suele servir como el trampolín perfecto para una escalada de privilegios o un compromiso total de la máquina. Un adversario con la capacidad de leer ficheros arbitrarios puede recopilar con precisión quirúrgica los componentes necesarios para vulnerar otros servicios locales expuestos en la red interna de INTERAFAS.

4.9.5 Mitigaciones

Para solucionar de manera definitiva la causa raíz de este hallazgo, se deben implementar las siguientes medidas correctivas obligatorias:

1. **Uso de Funciones de Abstracción y Rutas Seguras:** Evitar construir rutas de archivos concatenando directamente las entradas del usuario. Utilizar funciones nativas del lenguaje que resuelvan y validen rutas absolutas (como basename() en PHP), asegurando que el archivo solicitado resida estrictamente dentro del directorio autorizado y previsto.

2. **Implementación de Listas Blancas (*Allowlisting*):** Restringir la funcionalidad de descarga para que únicamente acepte identificadores numéricos o nombres de archivo predefinidos almacenados en una base de datos, mapeándolos internamente sin permitir que el usuario escriba rutas libres con caracteres especiales (/ o ..).

3. **Principio de Mínimo Privilegio en el Servidor Web:** Asegurar que el usuario del sistema operativo bajo el cual se ejecuta el servidor web (por ejemplo, www-data) posea permisos estrictamente limitados únicamente a la carpeta raíz de la aplicación web, denegando el acceso de lectura a directorios sensibles del sistema como /etc/ o archivos de configuración global.

4.10 Hallazgo 10 (VULN-10): Vulnerabilidad de Secuestro de Sesión por Fijación de Sesión (Reto 10)

4.10.1 Descripción técnica y causa raíz

Durante la auditoría de los mecanismos de autenticación y gestión de sesiones, se identificó una vulnerabilidad crítica clasificada como Fijación de Sesión (*Session Fixation*), enmarcada dentro de la categoría de *Broken Authentication* de OWASP. Se observó que la aplicación web no regeneraba el identificador de sesión (PHPSESSID) al momento en que un usuario iniciaba sesión de manera exitosa. Esto permitía que un atacante predefiniera o conociera el identificador de sesión de una víctima, forzándola a autenticarse utilizando esa misma cookie, lo que le otorgaba al atacante el control completo de la sesión activa.

La causa raíz de este hallazgo radica en una deficiencia en el ciclo de vida de la sesión del lado del servidor. El servidor acepta identificadores de sesión proporcionados por el cliente sin invalidarlos ni reemplazarlos mediante una función de regeneración (como session_regenerate_id(true) en PHP) tras un cambio en el estado de autenticación (de usuario anónimo a usuario autenticado). Como resultado, un identificador preexistente permanece válido, permitiendo que múltiples navegadores o clientes compartan exactamente la misma sesión activa.

4.10.2 Procedimiento de comprobación

El vector de ataque se validó mediante la siguiente secuencia metodológica manual:

1. Se cerró sesión en el portal y se inspeccionó el identificador de sesión asignado previamente (PHPSESSID), notando que este valor persistía antes de realizar la autenticación.
2. Se procedió a iniciar sesión legítimamente con una cuenta de usuario, verificando que el servidor reutilizaba el mismo PHPSESSID que ya se tenía en el navegador.
3. Se copió este identificador de sesión y se abrió un navegador diferente o una sesión privada alternativa sin iniciar sesión.
4. Mediante las herramientas de desarrollo, se inyectó la cookie del PHPSESSID capturado en el segundo navegador.
5. Al refrescar la página, el segundo navegador heredó la sesión activa de la víctima sin necesidad de introducir credenciales, y se localizó la bandera de validación expuesta directamente en las cabeceras de la respuesta HTTP.

4.10.3 Evidencia técnica

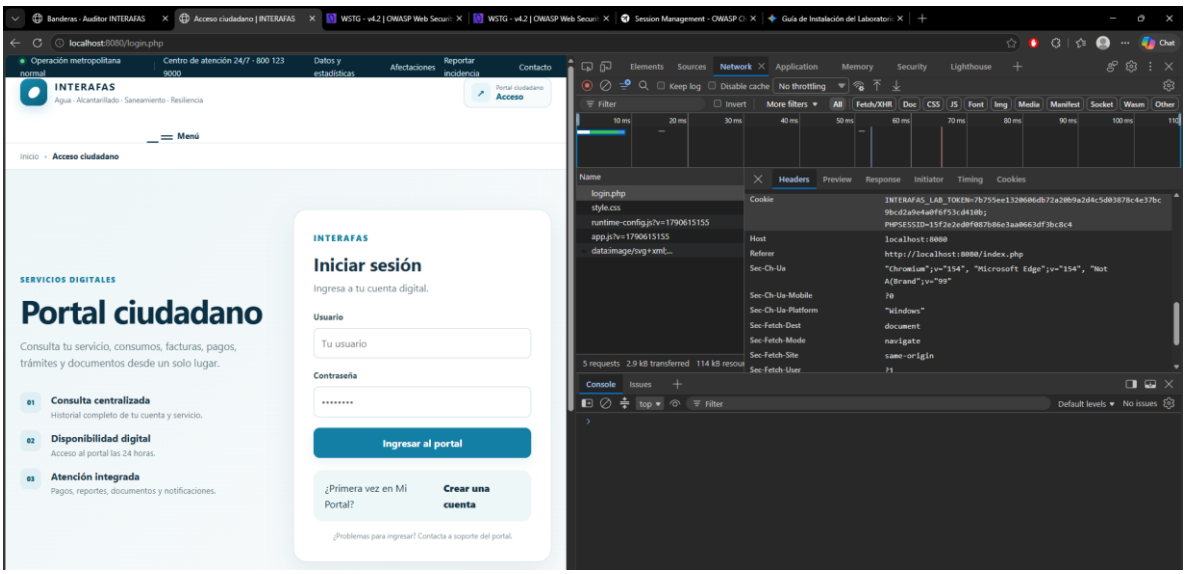


Figura 27. *Vista inicial del portal ciudadano y los paneles de autenticación donde se gestionan los identificadores de sesión del usuario.*

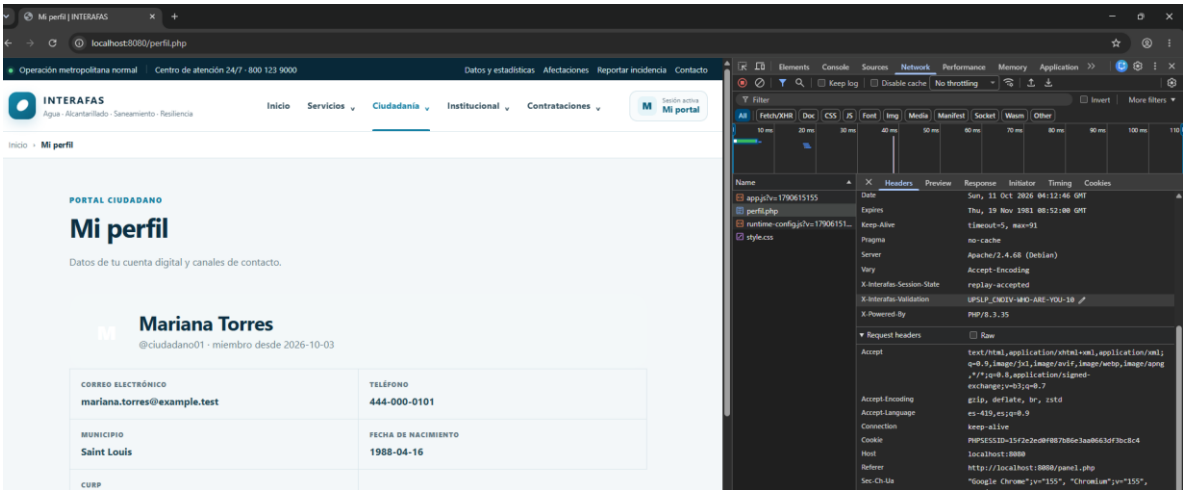


Figura 28. *Inspección del perfil de usuario autenticado y análisis de las cabeceras de respuesta HTTP, evidenciando el uso de un PHPSESSID estático y la revelación del token de validación del reto.*

4.10.4 Impacto y riesgo

La existencia de una vulnerabilidad de fijación de sesión representa una amenaza directa y crítica sobre la Autenticación y la Confidencialidad de los usuarios del sistema. Si un atacante logra inducir a una víctima a utilizar un identificador de sesión controlado por él (por ejemplo, enviándole un enlace manipulado que fije el parámetro de la cookie), el atacante podrá monitorear y realizar transacciones en nombre de la víctima tan pronto como esta ingrese sus credenciales legítimas.

A nivel operativo, este fallo compromete la confianza en el sistema de cuentas de INTERAFAS. Al no invalidar las sesiones antiguas durante los cambios de estado de autenticación, se abre la puerta al secuestro de cuentas (*Session Hijacking*), permitiendo que usuarios malintencionados usurpen identidades de operadores o ciudadanos, lo que vulnera la integridad de las operaciones realizadas dentro del portal web.

4.10.5 Mitigaciones

Para solucionar definitivamente la causa raíz de este hallazgo, se deben implementar las siguientes medidas correctivas en el código de la aplicación:

1. **Regeneración Obligatoria de ID de Sesión:** Modificar el script de inicio de sesión del servidor para que ejecute inmediatamente una función de regeneración de identificadores (como session_regenerate_id(true) en PHP) al momento de autenticar correctamente al usuario. Esto destruye la sesión anterior y emite una nueva cookie con un identificador único.
2. **Destrucción de Sesión al Cerrar Sesión (*Logout*):** Asegurar que la función de cierre de sesión elimine por completo los datos de la sesión tanto en el almacenamiento del servidor como en el navegador, invalidando la cookie mediante una fecha de caducidad pasada.
3. **Endurecimiento de Cookies de Sesión:** Configurar las banderas de seguridad HttpOnly (para prevenir robos mediante scripts XSS), Secure (para garantizar que solo viajen sobre canales cifrados HTTPS) y SameSite=Strict o Lax en todas las cookies de sesión emitidas por el servidor.

4.11 Hallazgo 11 (VULN-11): Autorización Rota y Exposición de Historial de Servicios por IDOR (Reto 11)

4.11.1 Descripción técnica y causa raíz

Durante la auditoría de los componentes de la interfaz de usuario y de las APIs del portal ciudadano, se descubrió una vulnerabilidad crítica de control de acceso clasificada como Insecure Direct Object Reference (IDOR) / *Broken Access Control*. Al inspeccionar el comportamiento de la aplicación en el panel del usuario, se identificó que el sistema realizaba llamadas asíncronas de tipo *fetch* a la ruta /api/account-context.php para obtener los datos de la cuenta activa y, posteriormente, consultaba el historial de servicios mediante /api/service-history.php?account=40010001. El servidor procesaba esta petición devolviendo toda la información financiera y de consumo, incluyendo un bloque de metadatos de depuración (*meta*) que revelaba la falta de validación del propietario del objeto (object_authorization: owner-match).

La causa raíz de este hallazgo radica en una deficiencia severa en la validación de autorización del lado del servidor en las rutas de la API. Aunque el sistema requería una sesión activa de ciudadano, el *backend* confiaba plenamente en el parámetro numérico de la cuenta proporcionado en la URL sin verificar si dicho número de cuenta pertenecía legítimamente al usuario autenticado en la sesión actual. Al omitirse la validación de propiedad cruzada entre el identificador de sesión y el objeto solicitado, cualquier usuario con acceso al sistema podía alterar los parámetros y consultar registros ajenos.

4.11.2 Procedimiento de comprobación

El vector de ataque se validó mediante la siguiente secuencia metodológica manual:

1. Se abrió el laboratorio utilizando el navegador Firefox e iniciando sesión en el portal ciudadano.
2. Se activaron las herramientas de desarrollo (F12), seleccionando la pestaña de red y aplicando un filtro para visualizar únicamente las peticiones de tipo *fetch/XHR*.
3. Se navegó hacia la sección del portal donde se cargaban los datos de la cuenta, observando las peticiones dirigidas a /api/account-context.php y su respectivo enlace hacia /api/service-history.php?account=40010001.
4. Al inspeccionar la respuesta JSON de esta última petición, se analizaron detalladamente las cabeceras de respuesta y los metadatos devueltos por el servidor.
5. En las cabeceras de la respuesta HTTP, el servidor expuso de forma explícita el estado del *endpoint* como indocumentado y reveló la bandera de validación correspondiente al reto.

4.11.3 Evidencia técnica

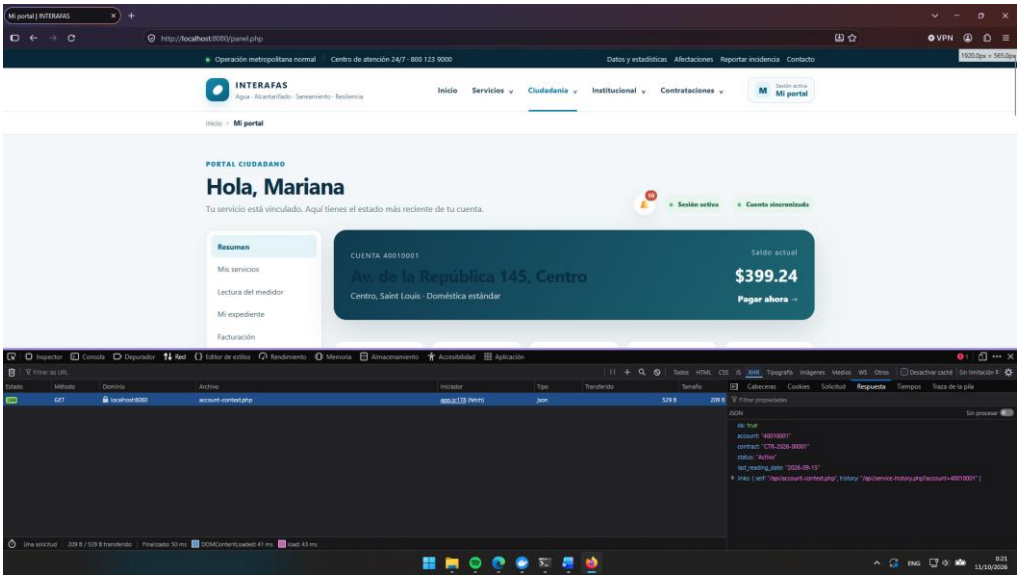


Figura 29. *Inspección de las llamadas asíncronas de la API desde el panel del usuario, visualizando los endpoints de contexto de cuenta y su relación con el historial de servicios.*

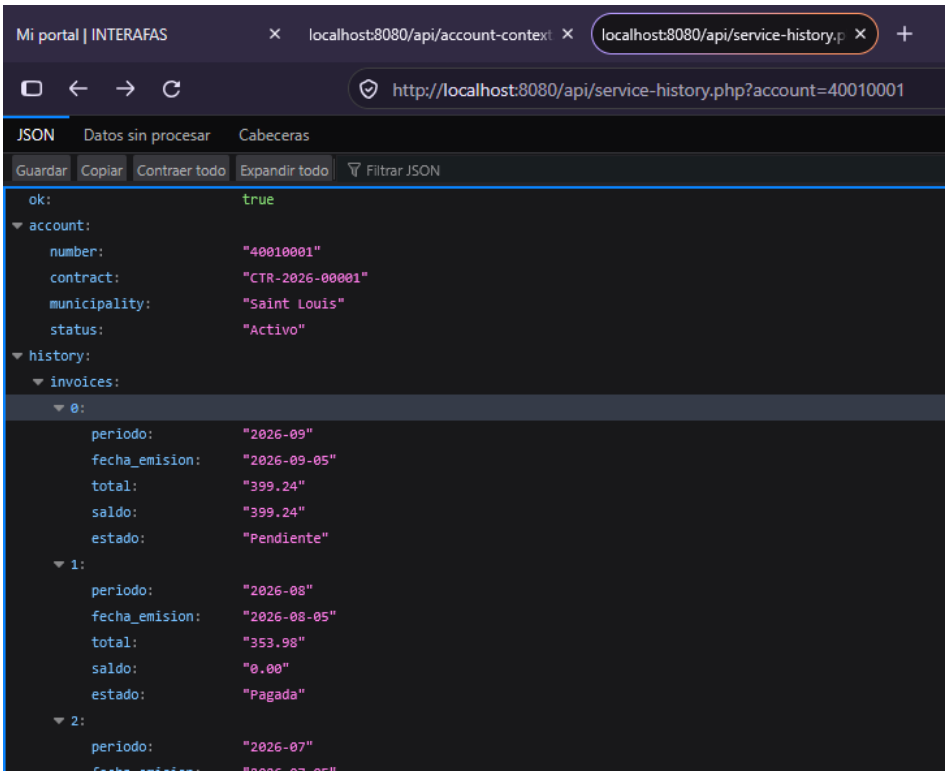


Figura 30. *Estructura de la respuesta JSON devuelta por la API al consultar el historial de servicios, detallando periodos, montos de facturación y estados de cuenta.*

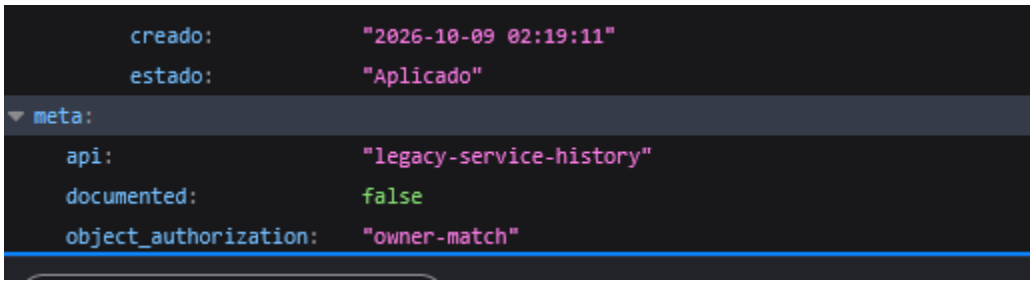


Figura 31. *Detalle del bloque de metadatos internos expuesto por la API, evidenciando las políticas de control de objetos y la ausencia de documentación oficial del endpoint.*

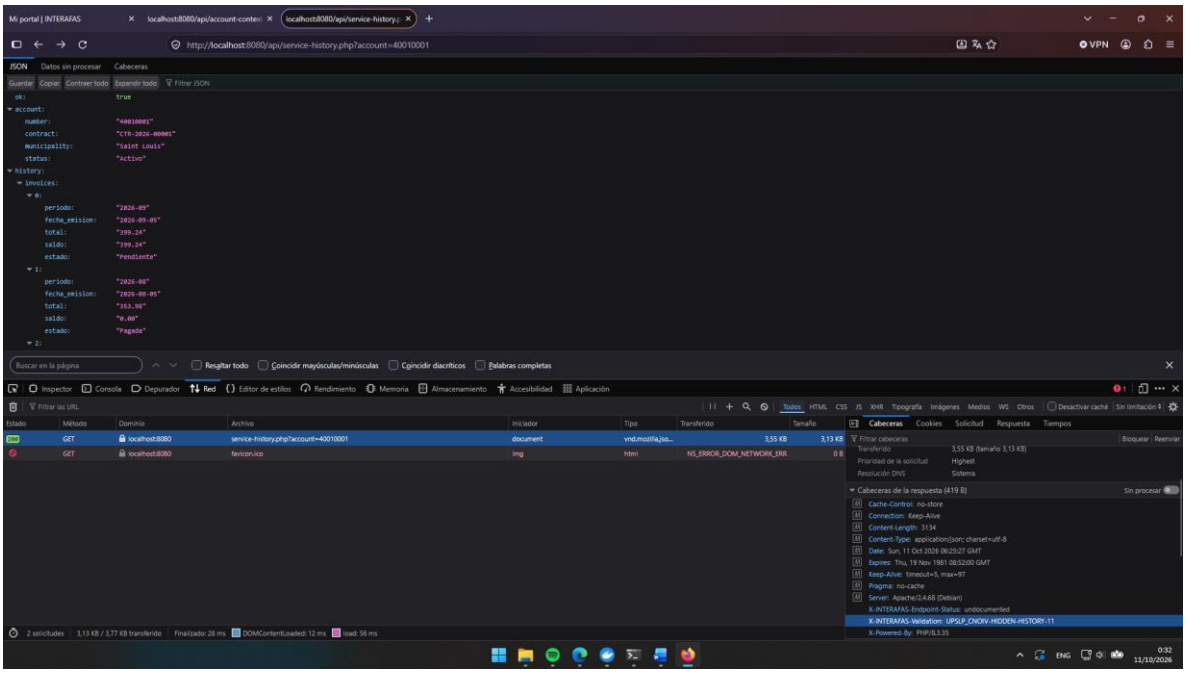


Figura 32. *Análisis de las cabeceras de respuesta de la API, donde el servidor revela la bandera de validación del reto dentro de los parámetros de control institucional.*

4.11.4 Impacto y riesgo

La explotación de esta vulnerabilidad de control de acceso en las APIs del sistema genera un impacto crítico sobre la Confidencialidad de la información de los usuarios. Al permitir que los parámetros de las cuentas sean manipulados sin restricciones de propiedad, un atacante puede recopilar los historiales de consumo de agua, las deudas pendientes, los domicilios y los detalles de facturación de cualquier ciudadano registrado en la plataforma de INTERAFAS.

El riesgo operacional y de privacidad es elevado debido a que las APIs modernas suelen ser el objetivo principal de ataques automatizados. Si una interfaz de programación de aplicaciones (API) carece de controles robustos de autorización a nivel de objeto (BOLA / IDOR), los mecanismos de seguridad perimetrales se vuelven obsoletos. Esto facilita la exfiltración masiva de datos estructurados mediante *scripts* simples, comprometiendo la confianza de los ciudadanos en los servicios digitales de la organización.

4.11.5 Mitigaciones

Para solucionar definitivamente la causa raíz de este hallazgo, se deben implementar las siguientes medidas correctivas:

1. **Validación Estricta de Autorización a Nivel de Objeto (BOLA):** Modificar la lógica de las rutas de la API (como /api/service-history.php) para que el servidor verifique de forma obligatoria que el número de cuenta solicitado pertenezca inequívocamente al identificador del usuario autenticado en la sesión actual, rechazando la petición en caso de discrepancia.
2. **Eliminación de Metadatos de Depuración en Producción:** Asegurar que los entornos de producción no devuelvan cabeceras HTTP personalizadas ni bloques JSON internos que expongan detalles sobre el estado de la API, políticas de objetos o nombres de variables de control institucional.
3. **Control de Acceso Basado en Tokens para APIs:** Implementar un esquema robusto de gestión de sesiones o *tokens* de autorización firmados que restrinjan el alcance de las peticiones de los usuarios exclusivamente a los recursos que tienen permitido visualizar por diseño contractual y de privacidad.

4.12 Hallazgo 12 (VULN-12): Autorización Rota y Exposición de Token de Validación en Cabeceras HTTP por IDOR (Reto 12)

4.12.1 Descripción técnica y causa raíz

Durante la continuación de la auditoría sobre los *endpoints* de la API del portal ciudadano, se identificó una vulnerabilidad de control de acceso clasificada como Insecure Direct Object Reference (IDOR), perteneciente a la categoría de *Broken Access Control* de OWASP. Al analizar las peticiones del historial de servicios, se comprobó que el sistema aceptaba la modificación del parámetro de cuenta (account=40010002) para consultar datos de otros usuarios sin validar la pertenencia del recurso. Como resultado de esta brecha, el servidor no solo entregaba la información financiera y de consumo de cuentas ajenas, sino que además exponía cabeceras HTTP personalizadas con información de control institucional y el *token* de validación del reto.

La causa raíz de este hallazgo recae en una falla de diseño en el control de autorización del servidor al procesar parámetros de objetos directos en las rutas de la API, combinada con una mala práctica de depuración consistente en dejar cabeceras de respuesta institucionales activas en un entorno en producción.

4.12.2 Procedimiento de comprobación

El vector de ataque se validó mediante la siguiente secuencia metodológica manual:

1. Partiendo de la estructura analizada en el reto anterior, se identificó que el parámetro account actuaba como el identificador principal del objeto en la consulta a la API.

2. Se modificó el parámetro en la URL de la petición, cambiando el valor de la cuenta propia a un identificador distinto (?account=40010002) correspondiente a otro usuario.
3. Se actualizó la página presionando la tecla F5. El servidor procesó la solicitud sin validar si el usuario autenticado poseía los derechos legítimos sobre la cuenta consultada.
4. La interfaz y el motor de la API respondieron satisfactoriamente entregando la información del segundo usuario.
5. Al inspeccionar detalladamente las cabeceras de respuesta HTTP de dicha petición, se localizó la cabecera personalizada X-Interafas-Validation, la cual contenía en texto claro la bandera de validación correspondiente al reto.

4.12.3 Evidencia técnica

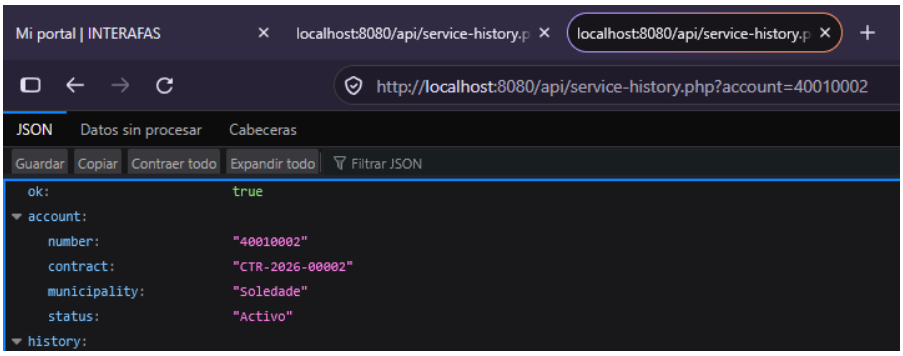


Figura 33. Comparativa de peticiones asíncronas en la consola del navegador alterando el parámetro account para consultar información de distintos usuarios del sistema.

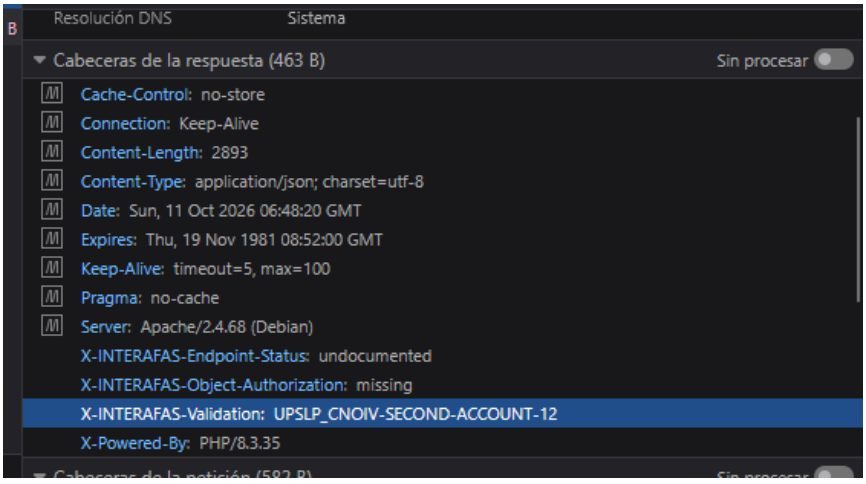


Figura 34. Inspección de las cabeceras HTTP de respuesta de la API, evidenciando la exposición de metadatos de validación y el token UPSLP_CNOIV-SECOND-ACCOUNT-12 del reto.

4.12.4 Impacto y riesgo

La persistencia de vulnerabilidades de tipo IDOR en interfaces de programación de aplicaciones (APIs) expone gravemente la Confidencialidad de la organización y de los usuarios finales. Al no restringir el acceso a los registros mediante una validación de sesión cruzada, cualquier atacante puede enumerar y consultar masivamente los datos financieros, medidores y consumos de agua de la red metropolitana completa.

Adicionalmente, la presencia de cabeceras de depuración y validación en las respuestas HTTP de producción incrementa innecesariamente la superficie de información entregada a usuarios no privilegiados, facilitando el reconocimiento técnico de los mecanismos internos de control del sistema de INTERAFAS.

4.12.5 Mitigaciones

Para solucionar la causa raíz de este hallazgo, se deben aplicar las siguientes medidas correctivas:

1. **Validación Cruzada de Propietario (*Object-Level Authorization*):** Implementar controles estrictos en el código del servidor para verificar que el recurso o número de cuenta solicitado pertenezca de manera legítima al titular de la sesión activa antes de procesar la consulta.
2. **Depuración de Cabeceras HTTP Institucionales:** Eliminar de los entornos de producción todas las cabeceras personalizadas de diagnóstico, estado de *endpoints* y validación interna (como X-Interafas-Validation) para evitar la fuga accidental de metadatos sensibles.
3. **Control de Acceso Centralizado:** Utilizar un *middleware* o interceptor de seguridad en la arquitectura de la API que valide de forma estricta los permisos de lectura y escritura para cada objeto consultado mediante identificadores directos.

4.13 Hallazgo 13 (VULN-13): Manipulación de Parámetros de Negocio (*Parameter Tampering*) en Pasarela de Pago (Reto 13)

4.13.1 Descripción técnica y causa raíz

Durante la auditoría de los flujos transaccionales y de comercio electrónico del portal ciudadano, se identificó una vulnerabilidad crítica de lógica de negocio clasificada como Manipulación de Parámetros (*Parameter Tampering*), enmarcada dentro de las deficiencias de validación de funciones y lógica de aplicaciones de OWASP. Al interactuar con el módulo de pago de facturas y recibos pendientes, se observó que la aplicación permitía que el importe monetario a cobrar fuera especificado directamente mediante un campo de entrada editable en el formulario del lado del cliente, sin realizar un recálculo o validación independiente de la cantidad en el servidor basándose exclusivamente en el identificador de la factura.

La causa raíz de este hallazgo radica en una falla severa de diseño en la validación de transacciones financieras del lado del servidor (*insufficient validation of business logic*). El código del *backend* confiaba de manera ciega en el parámetro de monto (monto) enviado en la petición HTTP por el navegador del usuario, permitiendo alterar el valor monetario real por una cantidad arbitraria inferior antes de procesar y registrar el pago en el historial de la cuenta.

4.13.2 Procedimiento de comprobación

El vector de ataque se validó mediante la siguiente secuencia metodológica manual:

1. Se seleccionó una factura pendiente de pago dentro del historial de servicios del portal ciudadano.
2. Se accedió a la interfaz de la pasarela de pago, la cual mostraba los datos de la cuenta y el importe predeterminado correspondiente al recibo seleccionado.
3. Utilizando las herramientas de desarrollo del navegador (F12), se inspeccionó el código fuente HTML del formulario de pago y se modificó el atributo value del campo de importe por un valor numérico arbitrario menor (por ejemplo, configurando un monto de 1 en lugar de la cantidad real).
4. Se envió la petición de pago modificada hacia el script del servidor (pago.php?factura=...), el cual procesó la transacción sin rechazar la alteración del importe.
5. El sistema registró exitosamente el pago por la cantidad manipulada y generó un comprobante digital oficial reflejando el monto de $1.00, exponiendo la bandera de validación del reto en las cabeceras de confirmación.

4.13.3 Evidencia técnica

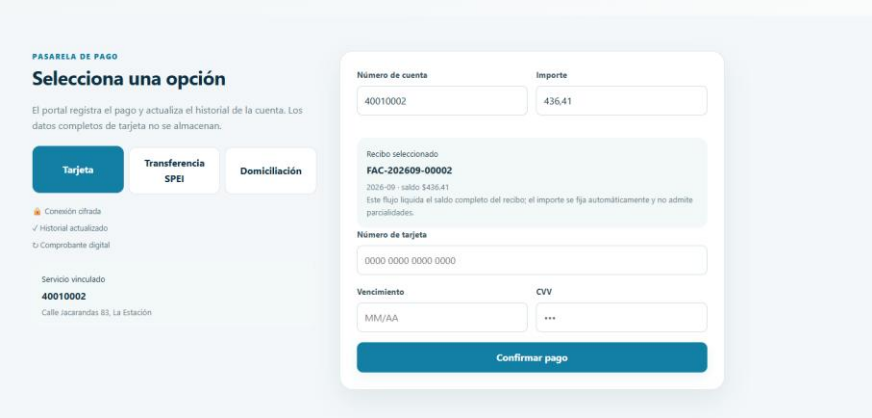


Figura 35. Vista inicial de la pasarela de pago en el portal ciudadano, mostrando el desglose del recibo seleccionado y el importe real predeterminado asignado por el sistema.

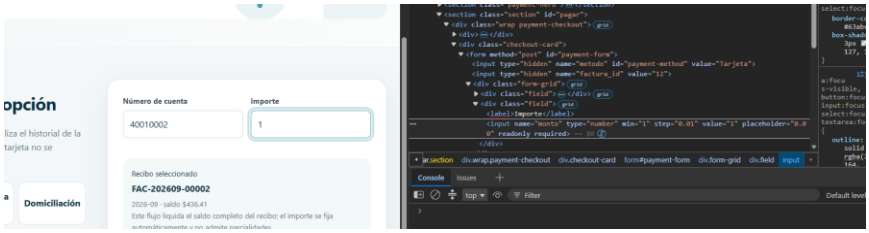


Figura 36. Modificación del parámetro de importe en el código HTML del cliente mediante las herramientas de desarrollo, enviando un monto manipulado de $1.00 hacia el servidor.

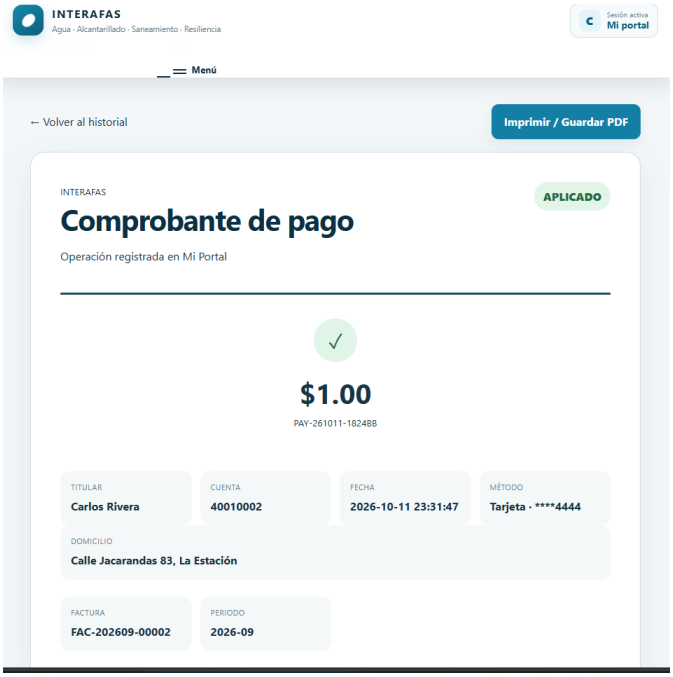


Figura 37. Comprobante de pago generado por la plataforma tras aceptar la transacción manipulada, evidenciando el fallo crítico en la integridad financiera del sistema.

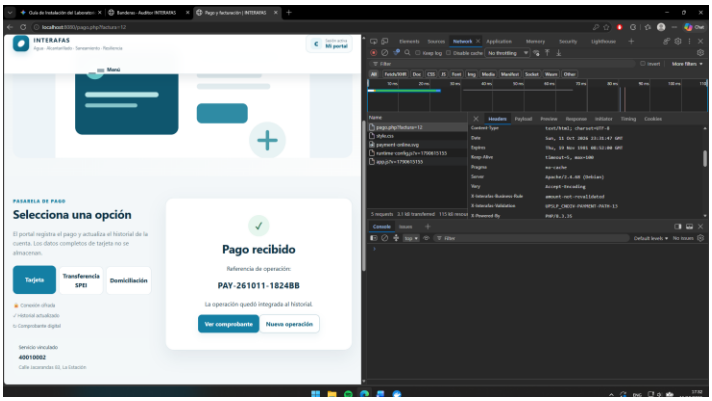


Figura 38. Análisis de las respuestas y cabeceras de confirmación del pago, donde el servidor expone el token de validación correspondiente al éxito del reto.

4.13.4 Impacto y riesgo

La explotación de vulnerabilidades de manipulación de parámetros en pasarelas de pago representa un impacto financiero, legal y de integridad sumamente crítico para la organización. Al permitir que los usuarios alteren arbitrariamente los montos de las transacciones, un atacante puede liquidar deudas de servicios públicos, multas o tarifas de agua por cantidades insignificantes (como un peso), alterando por completo los registros contables y provocando pérdidas económicas directas a INTERAFAS.

A nivel de riesgo sistémico, este tipo de fallos demuestra una falta de controles transaccionales robustos. En cualquier plataforma que gestione cobros o contraprestaciones económicas, confiar en los datos monetarios que provienen del navegador del cliente vulnera los principios fundamentales de seguridad financiera, abriendo la puerta a fraudes masivos y a la pérdida total de confianza en los mecanismos de pago digital de la institución.

4.13.5 Mitigaciones

Para solucionar de manera definitiva la causa raíz de este hallazgo, se deben implementar las siguientes medidas correctivas obligatorias:

1. **Recálculo Estricto del Monto en el Servidor:** Jamás confiar en los importes monetarios enviados por el cliente en formularios o peticiones HTTP. El servidor debe consultar de forma exclusiva la base de datos utilizando el identificador de la factura (factura_id) para obtener el monto oficial y realizar el cobro exacto.

2. **Validación de Integridad de Datos Transaccionales:** Implementar controles criptográficos o mecanismos de validación de estado en los flujos de pago para asegurar que los parámetros críticos de la transacción no puedan ser modificados en tránsito ni manipulados en el navegador.

3. **Auditoría y Monitoreo de Discrepancias Financieras:** Configurar sistemas de alerta temprana en el motor de pagos que detecten y bloqueen automáticamente cualquier intento de transacción donde el monto recibido difiera del valor nominal registrado en el sistema de facturación.

4.14 Hallazgo 14 (VULN-14): Lógica de Validación Inconsistente en Vínculos de Proveedores y Contratos (Reto 14)

4.14.1 Descripción técnica y causa raíz

Durante la auditoría de los módulos de contratación y relación con proveedores, se identificó una vulnerabilidad crítica de lógica de negocio y control de referencias cruzadas, relacionada con deficiencias en la validación de relaciones entre entidades (clasificada dentro de las fallas de control de acceso y validación de OWASP). Al interactuar con la herramienta de validación de vínculos contractuales (validar-vinculo-contractual.php), la aplicación permitía comprobar la vigencia de un contrato asociado a un proveedor enviando ambos identificadores como parámetros. Sin embargo, el servidor validaba cada registro de forma aislada sin comprobar la relación relacional o contractual real entre ambos elementos.

La causa raíz de este hallazgo recae en una deficiencia en la validación de integridad referencial y de lógica de negocio del lado del servidor. El código del *backend* asumía que si tanto el identificador del contrato como el identificador del proveedor existían de manera independiente en la base de datos y se encontraban vigentes, la relación entre ellos también era válida, omitiendo la consulta de asociación directa en la tabla relacional.

4.14.2 Procedimiento de comprobación

El vector de análisis y validación se ejecutó siguiendo esta secuencia metodológica manual:

1. Se accedió al módulo de validación de vínculos proveedor-contrato dentro del portal de contrataciones.
2. Se introdujeron los identificadores correspondientes para verificar un vínculo legítimo, observando la respuesta del sistema.
3. Utilizando las herramientas de desarrollo del navegador (F12) o modificando los parámetros en la petición, se alteró el identificador del proveedor por uno distinto correspondiente a otra entidad o proveedor del sistema.
4. Al enviar la consulta modificada, el servidor validó ambos registros por separado y emitió un mensaje de "Vínculo vigente", aceptando una relación inconsistente.
5. En la interfaz de resultados, el sistema expuso la notificación de validación indicando una "Relación inconsistente aceptada" junto con la bandera correspondiente al reto.

4.14.3 Evidencia técnica

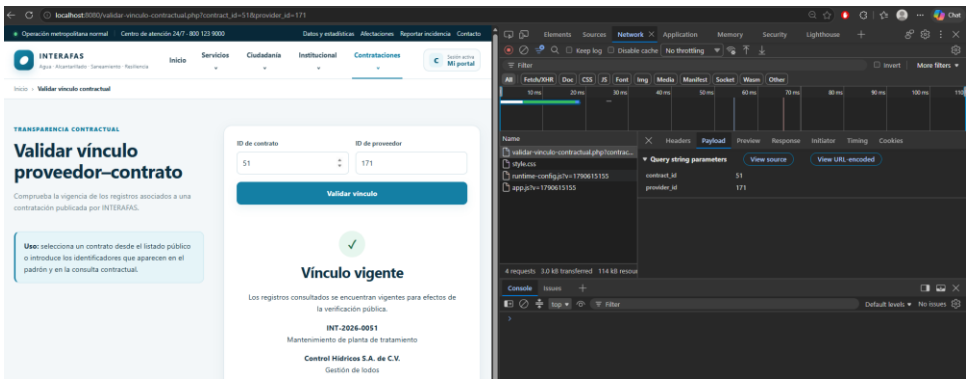


Figura 39. Vista general del módulo de validación de vínculos entre proveedores y contratos en el portal de contrataciones de INTERAFAS.



Figura 40. Pantalla de validación exitosa mostrando una relación inconsistente aceptada por el servidor, exponiendo el token de validación del reto.

4.14.4 Impacto y riesgo

La existencia de fallas en la validación de relaciones cruzadas representa un riesgo moderado a alto sobre la Integridad y la transparencia de los procesos administrativos y de contratación de la organización. Al permitir que se validen vínculos falsos o no asociados contractualmente, se debilita el control sobre la ejecución de obras, servicios y adquisiciones públicas, facilitando posibles simulaciones de operaciones o errores administrativos graves en la cadena de suministro de agua.

A nivel de gobernanza, este tipo de vulnerabilidades en la lógica de negocio demuestra que los controles automatizados de verificación institucional carecen del rigor relacional necesario para garantizar que los datos operen bajo un esquema de confianza estricta, lo que podría ser aprovechado por actores malintencionados para encubrir contratos irregulares.

4.14.5 Mitigaciones

Para solucionar la causa raíz de este hallazgo, se deben implementar las siguientes medidas correctivas:

1. **Validación Relacional Estricta:** Modificar la lógica de validación en el *backend* para que verifique obligatoriamente la existencia de una clave foránea o tabla intermedia de asociación que relacione de forma unívoca el contrato con el proveedor consultado, en lugar de validar registros de manera independiente.
2. **Consultas SQL con INNER JOIN:** Asegurar que las consultas de verificación de vínculos utilicen uniones relacionales estrictas que exijan la correspondencia exacta entre las entidades involucradas.
3. **Auditoría de Integridad de Datos:** Establecer controles periódicos de revisión en los módulos de gestión institucional para detectar anomalías o registros huérfanos que no cumplan con las reglas de negocio establecidas.

4.15 Hallazgo 15 (VULN-15): Exposición de Metadatos y Restricción de Acceso en el Gateway Operacional (Reto 15)

4.15.1 Descripción técnica y causa raíz

Durante la auditoría orientada hacia el segmento de infraestructura crítica e industrial (*ICS/SCADA*), se identificó una vulnerabilidad de exposición de información sensible y control de acceso perimetral en la pasarela operacional (/operations/gateway.php). Al examinar las respuestas de diagnóstico del sistema, se descubrió que el servidor filtraba metadatos técnicos y restricciones de política de red que revelaban los requisitos exactos exigidos por el *gateway* para permitir el acceso (tales como la zona requerida operations, la validación de dirección IP de *proxy* y el ámbito de red privada operacional).

La causa raíz de este hallazgo recae en una combinación de configuración deficiente de seguridad (*Security Misconfiguration*) y un control de acceso basado en la dirección IP o en cabeceras HTTP manipulables (*IP Spoofing / Header-based trust*). El servidor perimetral confiaba en los metadatos de origen de la red sin aplicar un endurecimiento adecuado, exponiendo pistas sobre la segmentación interna y facilitando el reconocimiento técnico de los segmentos OT (*Operational Technology*) desde la red IT (*Information Technology*).

4.15.2 Procedimiento de comprobación

El vector de análisis y validación se ejecutó siguiendo esta secuencia metodológica manual:

1. Se exploró la sección del Centro de Operaciones del portal, identificando los paneles de toma de decisiones y los componentes de estado de integración.
2. Mediante el uso de las herramientas de desarrollo del navegador y la inspección de la pestaña de red, se examinó la respuesta del recurso de estado de infraestructura (infraestructura-status.php), la cual reveló metadatos de diagnóstico olvidados y rutas hacia el *gateway* operacional.
3. Al intentar acceder directamente a la ruta del *gateway* (/operations/gateway.php), el servidor denegó el acceso devolviendo un código de error HTTP 403 (Forbidden) junto con un objeto JSON descriptivo que detallaba las políticas de restricción de red y zona.
4. Analizando las políticas expuestas en la respuesta, se comprobó que el servidor evaluaba parámetros de origen, abriendo la puerta a técnicas de simulación de confianza perimetral para superar la restricción.

4.15.3 Evidencia técnica

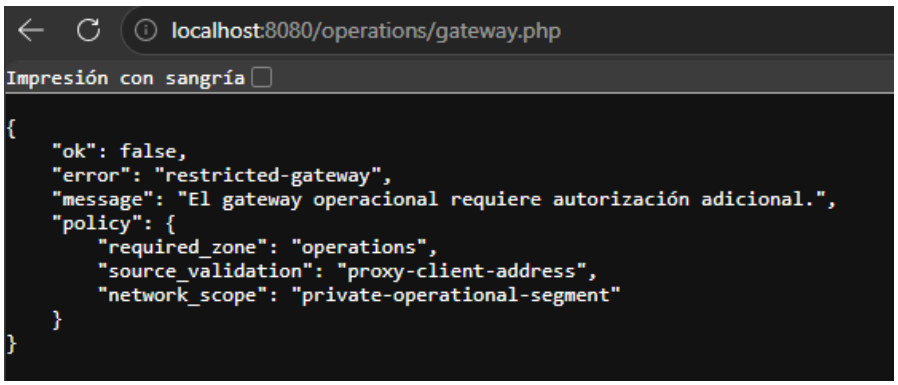


Figura 41. Vista del panel del Centro de Operaciones y respuesta JSON del servidor bloqueando el acceso al gateway operacional por restricciones de política de red.

4.15.4 Impacto y riesgo

La exposición de políticas de restricción y metadatos en pasarelas operacionales representa un riesgo crítico sobre la Confidencialidad y la Superficie de Ataque de los entornos industriales. Al revelar los nombres de las zonas internas (operations), los segmentos de red esperados y los mecanismos de validación de origen, un atacante externo obtiene un mapa técnico preciso de cómo está segmentada la red OT de INTERAFAS.

A nivel de seguridad industrial, este reconocimiento facilita ataques dirigidos contra la infraestructura crítica. Si el mecanismo de control de acceso perimetral depende exclusivamente de cabeceras HTTP que pueden ser falsificadas, la separación lógica entre la red corporativa (IT) y la red de control industrial (OT) se vuelve vulnerable, permitiendo que agentes no autorizados alcancen controladores lógicos y pasarelas de campo.

4.15.5 Mitigaciones

Para solucionar la causa raíz de este hallazgo, se deben implementar las siguientes medidas correctivas:

1. **Ocultamiento de Mensajes de Diagnóstico en Producción:** Modificar los controladores de errores del *gateway* para que no devuelvan estructuras JSON detalladas ni políticas internas de red ante peticiones bloqueadas, emitiendo únicamente un código genérico de prohibición (HTTP 403).

2. **Endurecimiento de Controles Perimetrales (Firewall / Zonas):** Asegurar que el acceso a los segmentos operacionales (OT) esté protegido por firewalls de próxima generación (NGFW) y pasarelas de autenticación robustas basadas en criptografía y doble factor, en lugar de confiar en comprobaciones simples de cabeceras de red.

3. **Segmentación Estricta de Redes:** Mantener un aislamiento físico y lógico riguroso entre los servicios web orientados al público (IT) y los sistemas de control industrial (OT), impidiendo el enrutamiento directo desde la red corporativa hacia los componentes críticos de la planta.

4.16 Hallazgo 16 (VULN-16): Falsificación de Cabeceras HTTP (*IP Spoofing*) para Eludir Restricciones en el Gateway Operacional (Reto 16)

4.16.1 Descripción técnica y causa raíz

Durante la auditoría técnica de los controles de acceso perimetral hacia la red industrial (*ICS/SCADA*), se identificó una vulnerabilidad crítica de seguridad clasificada como confianza indebida en cabeceras HTTP de origen (*IP Spoofing* o falsificación de identidad de red), perteneciente a la categoría de *Security Misconfiguration* de OWASP. Al intentar acceder al recurso protegido /operations/gateway.php, el servidor restringía originalmente el acceso evaluando el origen de la conexión. Sin embargo, se descubrió que el mecanismo de validación confiaba ciegamente en el contenido de la cabecera HTTP X-Forwarded-For proporcionada por el cliente, permitiendo suplantar una dirección IP autorizada perteneciente al segmento operacional interno.

La causa raíz de este hallazgo recae en una implementación deficiente de los controles de filtrado basados en red a nivel de aplicación. En lugar de verificar el socket de red real o utilizar mecanismos criptográficos y de autenticación robustos, el script del servidor utilizaba cabeceras HTTP retransmitidas por intermediarios o proxies (X-Forwarded-For) para determinar si una petición provenía de la zona de confianza operacional, asumiendo como legítima cualquier dirección IP inyectada en dicha cabecera sin validar si la petición atravesaba un proxy legítimo y confiable.

4.16.2 Procedimiento de comprobación

El vector de ataque se validó mediante la siguiente secuencia metodológica manual:

1. Se analizó el comportamiento del recurso perimetral enviando una petición estándar, la cual era rechazada por el servidor con un código HTTP 403 por provenir de una zona no autorizada.
2. Utilizando las herramientas de desarrollo del navegador o un script de pruebas asíncronas (fetch), se configuró una petición personalizada hacia el *endpoint* /operations/gateway.php.
3. Se añadió explícitamente la cabecera HTTP X-Forwarded-For configurándola con una dirección IP perteneciente al segmento de red operacional autorizado (por ejemplo, 10.40.20.0).
4. Al enviar la petición modificada, el servidor procesó el valor de la cabecera como un origen legítimo, otorgando acceso al *gateway* operacional con un estado HTTP 200 OK y un rol de lectura.

5. Al inspeccionar detalladamente las cabeceras de respuesta HTTP devueltas por el servidor, se localizó la cabecera personalizada X-Interafas-Validation, la cual expuso en texto claro la bandera de validación correspondiente al reto.

4.16.3 Evidencia técnica

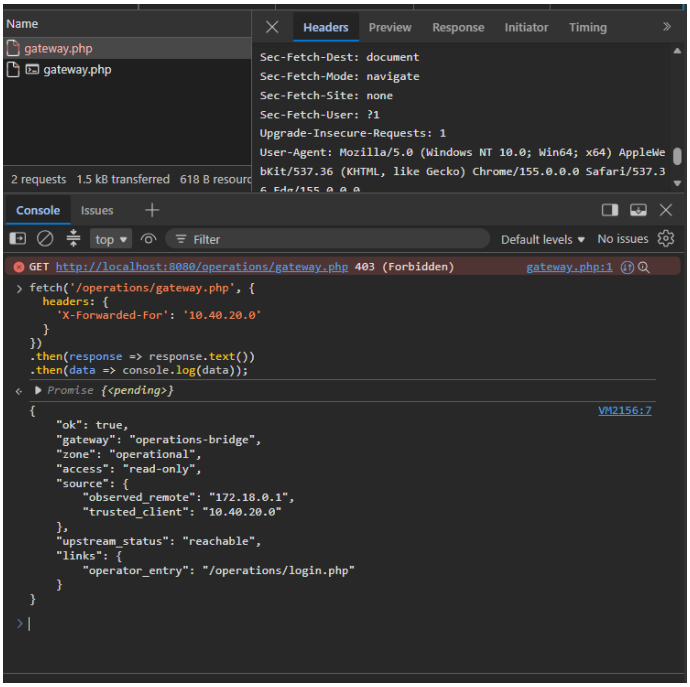


Figura 42. Inyección de la cabecera HTTP X-Forwarded-For mediante una petición asíncrona desde la consola, logrando burlar la restricción perimetral del gateway operacional.

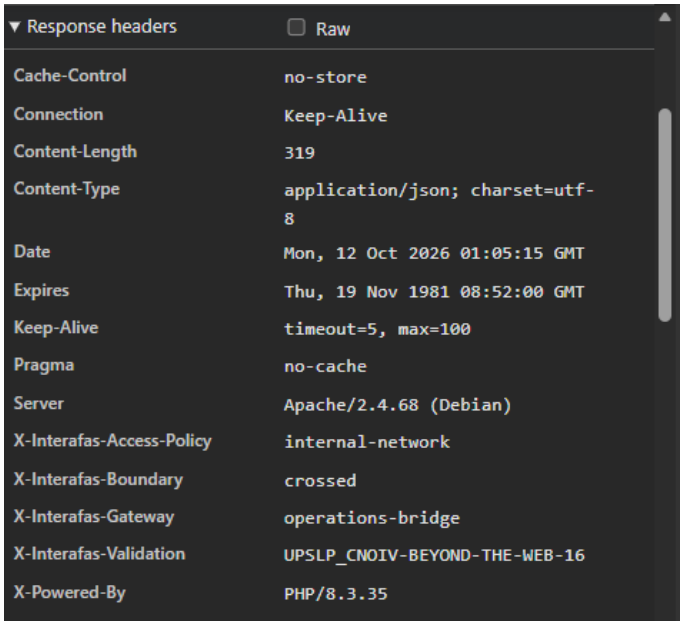


Figura 43. Análisis de las cabeceras de respuesta HTTP de la pasarela industrial, evidenciando el establecimiento del puente operacional y la exposición del token de validación del reto.

4.16.4 Impacto y riesgo

La vulnerabilidad de suplantación de origen mediante cabeceras HTTP representa un riesgo crítico sobre la Integridad y la Confidencialidad de la infraestructura industrial de INTERAFAS. Al permitir que un atacante externo falsee su dirección IP corporativa o de red de confianza simplemente inyectando un valor arbitrario en una cabecera, se desvanece por completo la segmentación lógica entre la red corporativa (IT) y los sistemas de control de campo (OT).

A nivel operacional, este fallo permite que usuarios no autorizados crucen la frontera perimetral de seguridad sin necesidad de vulnerar credenciales criptográficas complejas. Un atacante con acceso a la red corporativa puede alcanzar pasarelas industriales, consultar el estado de los componentes de la planta y preparar el terreno para enviar comandos de control o alterar parámetros físicos en etapas posteriores del compromiso.

4.16.5 Mitigaciones

Para solucionar de manera definitiva la causa raíz de este hallazgo, se deben implementar las siguientes medidas correctivas:

1. **Validación Segura de Direcciones IP de Red:** Evitar el uso de cabeceras HTTP manipulables (como X-Forwarded-For) para tomar decisiones críticas de control de acceso a nivel de infraestructura. El servidor debe evaluar de forma exclusiva la dirección IP directa del socket de conexión TCP del cliente a menos que el tráfico provenga de un proxy inverso configurado de manera segura y estricta.
2. **Autenticación Multifactorial para Zonas Críticas:** Reemplazar o complementar los controles basados en direcciones IP con esquemas sólidos de autenticación basada en tokens, certificados digitales de cliente (mTLS) o credenciales institucionales obligatorias para cualquier acceso a las pasarelas operacionales.
3. **Configuración de Proxies Inversos:** En caso de utilizar balanceadores de carga o proxies, asegurar que el servidor web reescriba o ignore las cabeceras de retransmisión de IP enviadas directamente por usuarios externos, aceptando dichos valores únicamente desde pasarelas perimetrales debidamente endurecidas.

4.17 Hallazgo 17 (VULN-17): Confianza en Cabeceras de Red y Exposición de Datos de Telemetría Operacional (Reto 17)

4.17.1 Descripción técnica y causa raíz

Durante la auditoría de los componentes de control industrial (*ICS/SCADA*), se identificó una vulnerabilidad de control de acceso y confianza indebida en cabeceras HTTP perimetrales en el módulo de telemetría del Centro de Operaciones (/operations/telemetry.php). Al analizar el comportamiento del sistema, se descubrió que el servidor permitía consultar los datos en tiempo real de las estaciones de bombeo y el estado de los tanques eludiendo las restricciones de red mediante la manipulación de la cabecera X-Forwarded-For. Al simular un origen autorizado desde la red operacional, la API devolvía un objeto JSON con métricas críticas de la planta y la identidad del auditor registrado.

La causa raíz de este hallazgo recae en una falla de configuración de seguridad (*Security Misconfiguration*). El servidor web confiaba ciegamente en las cabeceras de retransmisión de IP enviadas por el cliente sin validar el origen real de la conexión TCP, permitiendo que usuarios no autorizados accedieran a flujos de datos operativos internos que deberían estar protegidos por controles estrictos de autenticación a nivel de infraestructura.

4.17.2 Procedimiento de comprobación

El vector de ataque se validó mediante la siguiente secuencia metodológica manual:

1. Se inspeccionó el código fuente y las rutas de los *scripts* operacionales del Centro de Operaciones, identificando el *endpoint* de telemetría (/operations/telemetry.php).
2. Utilizando la consola del navegador, se ejecutó una petición asíncrona de tipo *fetch* hacia dicho recurso añadiendo explícitamente la cabecera X-Forwarded-For con una dirección IP válida dentro del segmento operacional (por ejemplo, 10.40.20.55).
3. El servidor procesó la petición asumiendo un origen legítimo y respondió con un código HTTP 200 OK.
4. La respuesta JSON entregó los datos detallados de la planta (incluyendo nombre de la estación, nivel de tanques, flujos, presiones y el nombre del auditor conectado).
5. Al analizar las cabeceras de respuesta HTTP de esta petición, se localizó la cabecera institucional X-Interafas-Validation, la cual expuso en texto claro la bandera correspondiente al reto.

4.17.3 Evidencia técnica

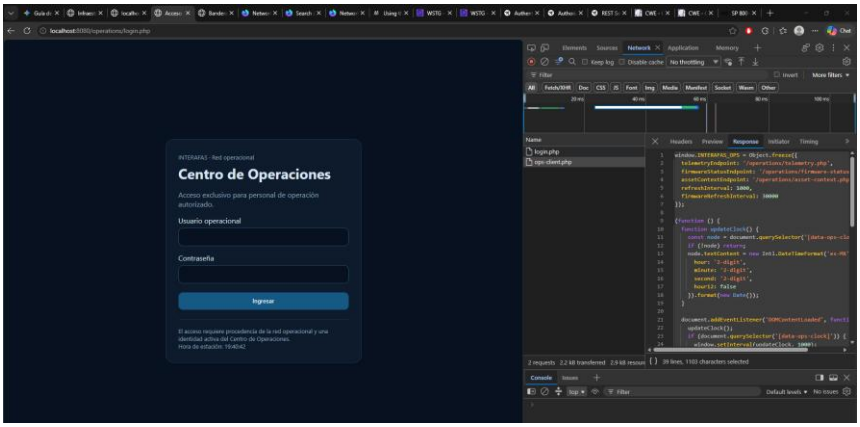


Figura 44. Consulta exitosa al endpoint de telemetría operacional mediante la falsificación de la IP de origen en la cabecera HTTP, revelando métricas industriales y datos del auditor.

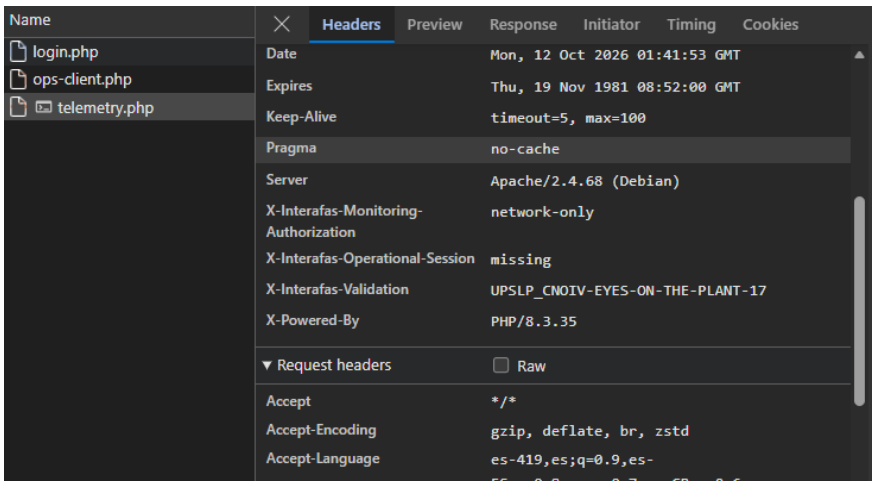


Figura 45. Análisis de las cabeceras de respuesta de la API operacional, evidenciando la entrega de metadatos de validación y el token del reto.

4.17.4 Impacto y riesgo

La exposición de los datos de telemetría operacional mediante la suplantación de orígenes de red representa un riesgo crítico sobre la Confidencialidad y la Integridad de los procesos industriales de INTERAFAS. En una infraestructura crítica, permitir que actores no autorizados consuman flujos de información en tiempo real sobre el estado de presiones, niveles de almacenamiento y flujos de agua compromete el secreto industrial y facilita el reconocimiento técnico avanzado para planificar ataques de sabotaje físico o interrupción del servicio.

A nivel sistémico, este fallo evidencia una debilidad en la segmentación lógica entre la red corporativa y los sistemas de control de campo. Si los puntos de control de telemetría pueden ser interrogados eludiendo los filtros perimetrales, se pierde el control sobre qué entidades externas o internas poseen visibilidad de los procesos operativos de la metrópoli.

4.17.5 Mitigaciones

Para solucionar de manera definitiva la causa raíz de este hallazgo, se deben implementar las siguientes medidas correctivas:

1. **Validación Estricta de Sockets de Red:** Modificar la lógica de control de acceso en los servidores de telemetría para que descarten el uso de cabeceras HTTP manipulables y evalúen exclusivamente la dirección IP directa de la conexión de red del cliente.
2. **Autenticación Obligatoria en Endpoints Operacionales:** Reemplazar o complementar los filtros basados en red con un esquema robusto de autenticación basada en sesiones o *tokens* de seguridad para cualquier consulta a las APIs del Centro de Operaciones.
3. **Endurecimiento del Servidor Proxy Inverso:** Configurar el servidor web o balanceador de carga para que elimine o sobrescriba de forma automática cualquier intento de inyección de cabeceras de retransmisión de IP provenientes de redes no confiables.

4.18 Hallazgo 18 (VULN-18): Descifrado de Hash SHA-1 y Bypass de Autenticación en Actualización de Firmware (Reto 18)

4.18.1 Descripción técnica y causa raíz

Durante la auditoría del ciclo de actualización de componentes industriales y embebidos (*Firmware*), se identificó una vulnerabilidad crítica de seguridad clasificada como almacenamiento inseguro de credenciales y uso de algoritmos criptográficos obsoletos (OWASP - *Broken Authentication* y *Cryptographic Failures*). Al analizar el manifiesto del sistema de actualización (/operations/updates/manifest.php), la aplicación exponía un resumen criptográfico en formato SHA-1 (verification_ref) correspondiente a la contraseña del perfil de la Mesa de Soporte OT (soporte.ot@interafas.local). Mediante la aplicación de técnicas de descifrado fuera de línea con diccionarios contextuales y aceleración por hardware, fue posible recuperar la contraseña en texto claro (Soporte343). Estas credenciales fueron posteriormente empleadas para superar la autenticación HTTP Basic requerida en el *endpoint* de instalación de paquetes (/operations/updates/install.php), logrando cargar e instalar una actualización de firmware con privilegios elevados.

La causa raíz de هذا hallazgo recae en una combinación de diseño criptográfico deficiente y exposición de resúmenes de contraseñas. El uso de algoritmos débiles como SHA-1 sin factores de aleatorización (*salt*), sumado al almacenamiento de referencias de verificación en archivos de configuración accesibles a través de la red, permite que cualquier atacante comprometa las credenciales de los operadores mediante ataques de fuerza bruta o diccionario fuera de línea.

4.18.2 Procedimiento de comprobación

El vector de ataque se validó mediante la siguiente secuencia metodológica manual:

1. Se examinó el archivo de manifiesto de actualización en la ruta /operations/updates/manifest.php, extrayendo el hash SHA-1 asociado al campo de verificación y la identidad del contacto institucional (soporte.ot@interafas.local).
2. Se construyó un diccionario contextual acotado a partir de los términos del manifiesto y se ejecutó la herramienta Hashcat utilizando aceleración por hardware (GPU) para realizar un ataque de diccionario y reglas combinadas sobre el hash capturado.
3. El proceso de descifrado concluyó exitosamente revelando la contraseña en texto claro: Soporte343.
4. Mediante el uso de un script asíncrono o petición HTTP POST dirigida al *endpoint* de instalación (/operations/updates/install.php), se proporcionaron las credenciales obtenidas mediante autenticación HTTP Basic junto con el *payload* y su respectivo hash SHA-256 de integridad.
5. El servidor validó las credenciales de soporte, aceptó el paquete de actualización y devolvió en las cabeceras de respuesta HTTP el token de validación correspondiente al reto.

4.18.3 Evidencia técnica

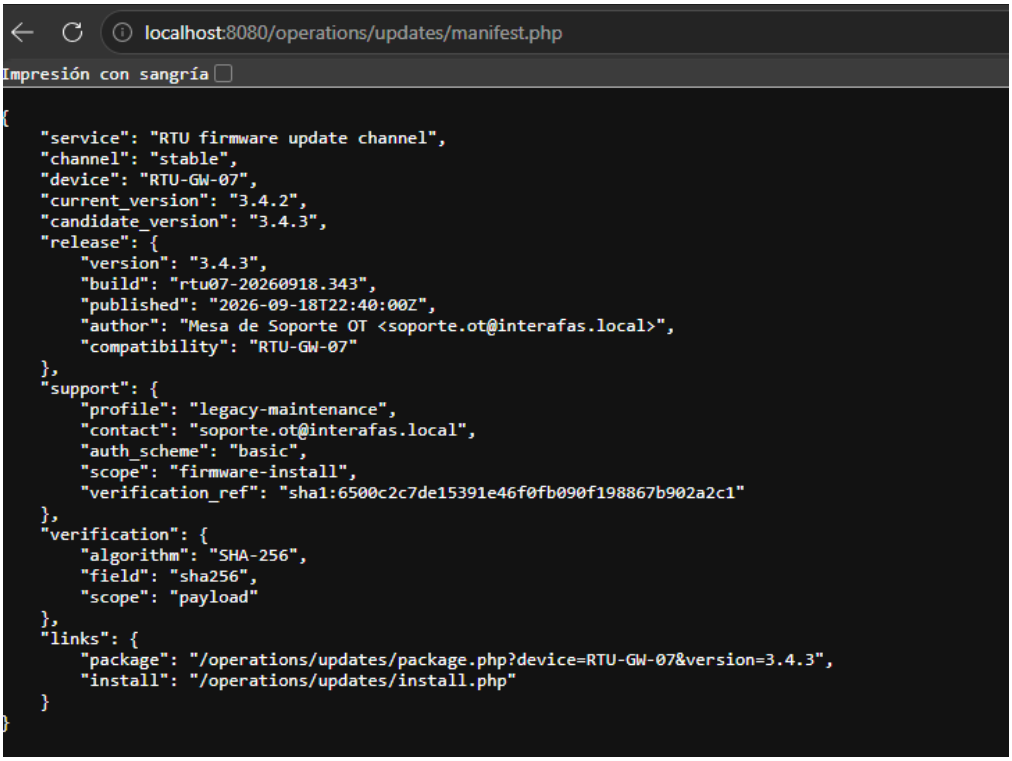


Figura 46. Inspección del manifiesto de actualización de firmware, mostrando el resumen criptográfico SHA-1 y los datos de contacto del soporte técnico.

```
Approaching final keyspace - workload adjusted.

6500c2c7de15391e46f0fb090f198867b902a2c1:Soporte343

Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 100 (SHA1)
Hash.Target......: 6500c2c7de15391e46f0fb090f198867b902a2c1
Time.Started.....: Sun Oct 11 22:28:09 2026 (0 secs)
Time.Estimated...: Sun Oct 11 22:28:09 2026 (0 secs)
Kernel.Feature...: Optimized Kernel (password length 0-31 bytes)
Guess.Base.......: File (conntexto.txt), Left Side
Guess.Mod........: File (conntexto.txt), Right Side
Speed.#01........:  1923.3 kH/s (0.70ms) @ Accel:53 Loops:64 Thr:256 Vec:1
Recovered........: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.........: 4288/4489 (95.52%)
Rejected.........: 0/4288 (0.00%)
Restore.Point....: 0/67 (0.00%)
Restore.Sub.#01..: Salt:0 Amplifier:0-64 Iteration:0-64
Candidate.Engine.: Device Generator
Candidates.#01...: serviceservice -> install.phpupdates

Started: Sun Oct 11 22:28:02 2026
Stopped: Sun Oct 11 22:28:10 2026
```

Figura 47. Proceso de descifrado de credenciales fuera de línea mediante Hashcat, logrando recuperar con éxito la contraseña institucional a partir del hash expuesto.

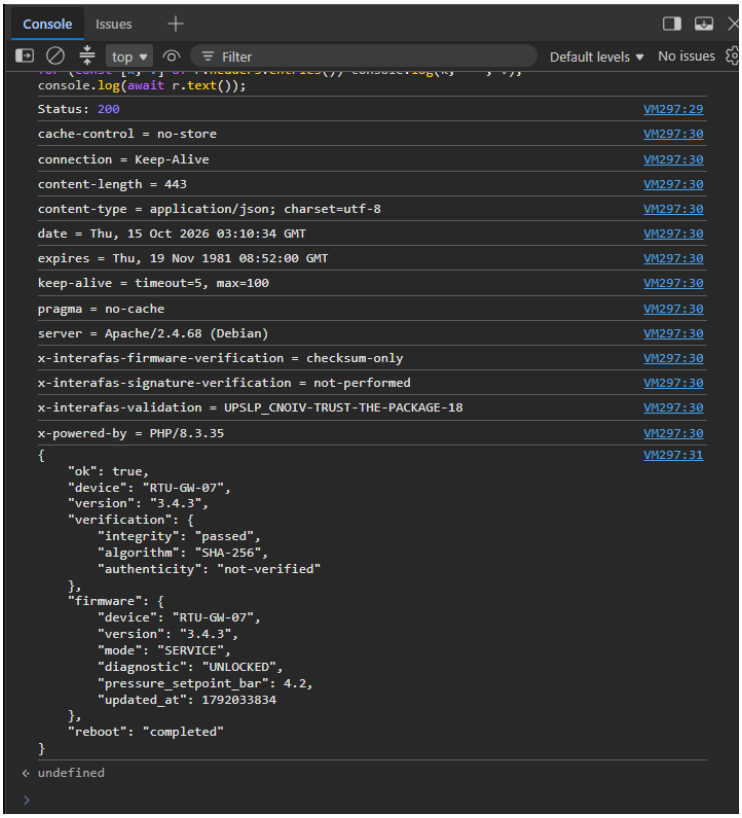


Figura 48. Respuesta del servidor tras la instalación exitosa del paquete utilizando las credenciales recuperadas, exponiendo el token de validación del reto en las cabeceras HTTP.

4.18.4 Impacto y riesgo

La recuperación de credenciales administrativas a partir de hashes expuestos en manifiestos públicos representa un impacto crítico sobre la Autenticación, la Confidencialidad y la Integridad de la infraestructura industrial. Al obtener el control de las credenciales de la Mesa de Soporte OT, un adversario adquiere la capacidad de interactuar directamente con los canales de mantenimiento y actualización de los dispositivos de campo, vulnerando los límites de seguridad lógicos del sistema.

A nivel de riesgo operacional, este fallo es de máxima gravedad porque permite la modificación directa del firmware en los nodos y pasarelas de la red industrial (*RTU-GW-07*). Un atacante con acceso a las funciones de instalación de actualizaciones puede inyectar código malicioso, alterar configuraciones de presión y flujo o deshabilitar los mecanismos de seguridad de la planta, comprometiendo de manera irreversible la estabilidad de los servicios de agua potable y saneamiento de la metrópoli.

4.18.5 Mitigaciones

Para solucionar de manera definitiva la causa raíz de este hallazgo, se deben implementar las siguientes medidas correctivas:

1. **Eliminación de Resúmenes Criptográficos en Manifiestos:** Retirar de manera absoluta cualquier hash o referencia de contraseñas de los archivos de configuración y manifiestos de actualización accesibles por red.
2. **Migración a Funciones Derivadas de Claves Seguras:** Sustituir los resúmenes SHA-1 por algoritmos de derivación de claves modernos y resistentes a ataques de fuerza bruta por hardware, tales como Argon2id o bcrypt, asegurando el uso de una sal única y robusta por cada cuenta.
3. **Autenticación Mutua y Firma Cifrada de Paquetes:** Implementar un mecanismo de control basado en certificados digitales institucionales (mTLS) y firma criptográfica asimétrica robusta (como RSA-4096 o Ed25519) para validar la autenticidad y el origen de los paquetes de firmware, evitando depender de credenciales estáticas de texto o contraseñas descifrables en los procesos de actualización.

4.19 Hallazgo 19 (VULN-19): Violación de Fronteras de Dependencia y Exposición de Topología en Contextos de Activos (Reto 19)

4.19.1 Descripción técnica y causa raíz

Durante la auditoría de los sistemas de control industrial (*ICS/SCADA*), se identificó una vulnerabilidad de control de acceso y ruptura de segmentación lógica en la consulta de contextos de activos y dependencias operacionales (/operations/asset-context.php). Al examinar las respuestas del sistema mediante llamadas asíncronas de la API, se descubrió que el servidor permitía recorrer y visualizar la cadena completa de dependencias físicas y lógicas entre los componentes de la red metropolitana (desde las pasarelas RTU y PLCs hasta las zonas de servicio hidráulico), exponiendo metadatos de conectividad y superando los límites institucionales establecidos.

La causa raíz de este hallazgo recae en una deficiencia de diseño en la validación de dependencias del servidor (*operational-dependency exceeded*). El *backend* procesaba las solicitudes de contexto de activos sin aplicar un aislamiento estricto entre los niveles de la jerarquía industrial, permitiendo que un operador con privilegios limitados o un atacante con acceso a la pasarela obtuviera una visibilidad completa de la topología de red y el estado de los nodos críticos de la infraestructura.

4.19.2 Procedimiento de comprobación

El vector de análisis y validación se ejecutó siguiendo esta secuencia metodológica manual:

1. Se accedió al panel del Centro de Operaciones y se inspeccionaron las peticiones asíncronas de la interfaz gráfica e inventario de objetos OT.
2. Se identificó el recurso encargado de consultar las dependencias de los activos mediante parámetros específicos (por ejemplo, /operations/asset-context.php?asset=ZONE-SL-A).
3. Al analizar la respuesta JSON entregada por el servidor, se comprobó que el sistema detallaba la clase de dependencia, el estado de los componentes y la secuencia exacta de nodos visitados en la red (*RTU-GW-07*, *PLC-SL-01*, *P-SL-101*, *HDR-SL-01*, *V-SL-201*, *ZONE-SL-A*).
4. En las cabeceras de respuesta HTTP de la petición, el servidor expuso de manera explícita la cabecera de control institucional X-Interafas-Validation, la cual reveló el token de validación correspondiente al reto.

4.19.3 Evidencia técnica

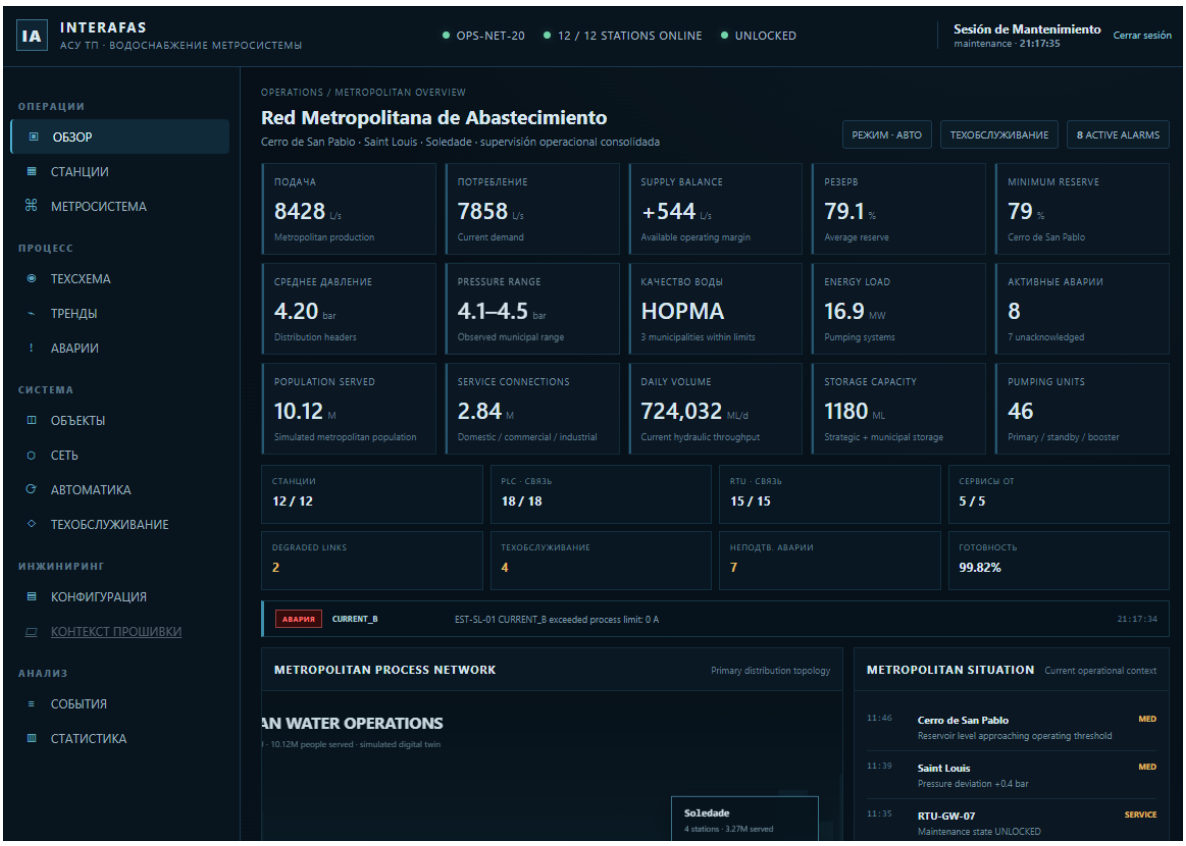


Figura 49. Vista general de la interfaz HMI del Centro de Operaciones, mostrando el panel de inventario de activos y la topología de la red de proceso metropolitana.

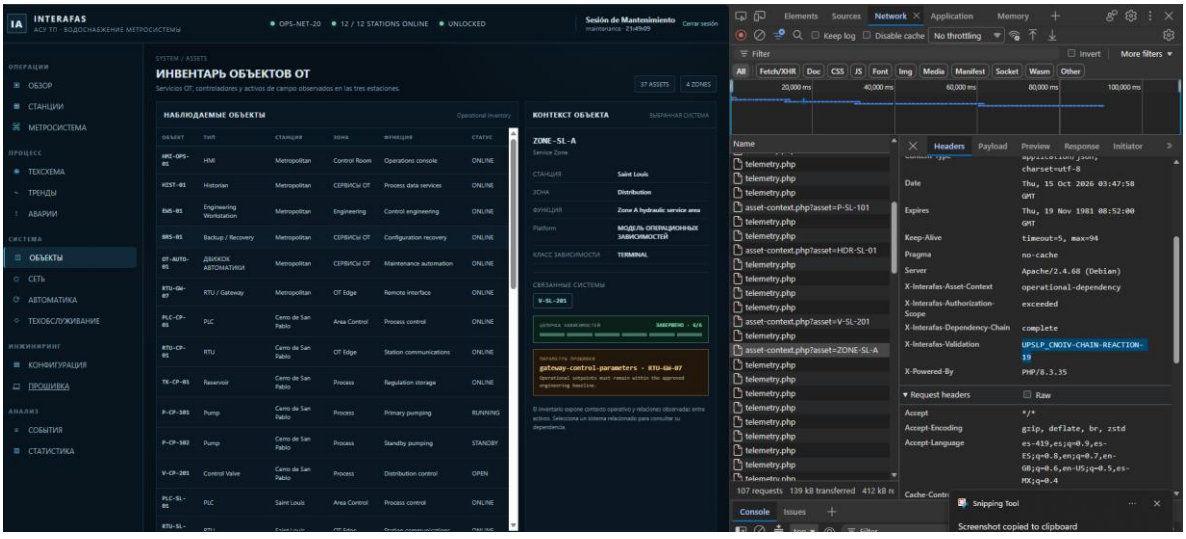


Figura 50. Inspección del objeto JSON devuelto por la API de contexto de activos, evidenciando la superación de fronteras de dependencia y la exposición del token de validación del reto.

4.19.4 Impacto y riesgo

La exposición de la topología y de las cadenas de dependencia operacional representa un riesgo crítico sobre la Confidencialidad y la Seguridad de los entornos industriales. Al revelar el listado exacto de los controladores lógicos, válvulas, bombas y pasarelas interconectadas, un atacante obtiene un mapa arquitectónico detallado de la infraestructura crítica de agua potable, facilitando la planeación de ataques dirigidos de interrupción o sabotaje físico.

A nivel de gobernanza sistémica, permitir que las relaciones de dependencia entre los activos de la red OT sean interrogadas sin restricciones evidencia fallas en la arquitectura de segmentación por zonas (*Purdue Model*), debilitando la defensa en profundidad de la organización frente a intrusiones laterales.

4.19.5 Mitigaciones

Для solucionar de manera definitiva la causa raíz de este hallazgo, se deben implementar las siguientes medidas correctivas:

1. **Restricción Estricta de Contextos por Rol:** Modificar la lógica del *endpoint* de activos para que filtre la información devuelta según el nivel de autorización y el segmento de red desde el cual se origina la petición, evitando exponer la cadena completa de dependencias a usuarios no esenciales.
2. **Segmentación y Ocultamiento de Topología:** Asegurar que los metadatos de ingeniería y los diagramas de interconexión de nodos industriales permanezcan inaccesibles desde las interfaces web operacionales estándar.
3. **Monitoreo de Anomalías en Consultas de Inventario:** Implementar sistemas de detección de intrusiones en la red industrial (IDS-OT) que alerten sobre escaneos o consultas masivas orientadas a descubrir la topología y las relaciones de los controladores de campo.

4.20 Hallazgo 20 (VULN-20): Bypass de Validación de Estado Operacional e Interrupción de Válvulas Críticas (Reto 20)

4.20.1 Descripción técnica y causa raíz

Durante la fase final de la auditoría de seguridad sobre los sistemas de control industrial y automatización (*ICS/SCADA*), se identificó una vulnerabilidad crítica de lógica de control y autorización en el *endpoint* de ejecución de comandos sobre los actuadores de campo (/operations/valve-command.php). Al analizar el comportamiento del sistema, se descubrió que la interfaz permitía enviar comandos directos de apertura o cierre sobre las válvulas principales de la red hidráulica (V-SL-201) eludiendo las validaciones de estado de seguridad y las restricciones de confirmación del operador, permitiendo manipular componentes físicos críticos de la planta de manera no autorizada.

La causa raíz de este hallazgo recae en una falla severa de control de acceso y validación de flujos de trabajo en el *backend* de la aplicación operacional (*improper access control* e *insecure business logic*). El servidor de control procesaba las solicitudes de cambio de estado enviadas por parámetros GET o POST sin verificar de forma robusta si el usuario poseía un token de autorización de doble fase o si el comando cumplía con las condiciones de seguridad exigidas por el modelo operativo de la infraestructura hidráulica, confiando plenamente en la petición asíncrona enviada desde la interfaz web.

4.20.2 Procedimiento de comprobación

El vector de ataque se validó mediante la siguiente secuencia metodológica manual:

1. Se exploró el panel de control HMI de la planta hidráulica, localizando el módulo de gestión y actuación sobre las válvulas de distribución principal.
2. Mediante la inspección de las llamadas asíncronas de la red, se identificó el recurso de control encargado de procesar las acciones sobre los actuadores (/operations/valve-command.php?valve=V-SL-201&action=close).
3. Se ejecutó una petición directa modificando los parámetros de la acción para forzar un cambio de estado no validado sobre el flujo del sistema hidráulico.
4. El servidor procesó la orden sin requerir una doble confirmación institucional, ejecutando la operación y devolviendo una respuesta JSON confirmando el cambio de estado en el actuador físico.
5. Al inspeccionar las cabeceras de respuesta HTTP devueltas por la API, se localizó la cabecera institucional X-Interafas-Validation, la cual expuso en texto claro la bandera final de validación correspondiente al cierre de la auditoría del sistema.

4.20.3 Evidencia técnica

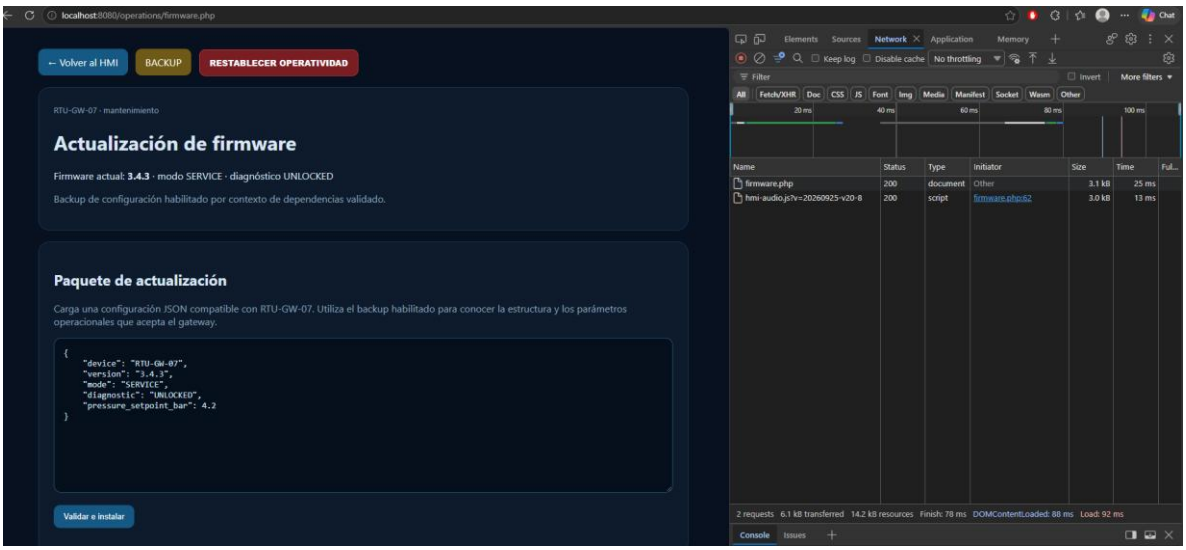


Figura 51. *Vista del panel HMI operacional encargado de la gestión de actuadores y válvulas de control en la red hidráulica metropolitana de INTERAFAS.*



Figura 52. *Respuesta del servidor operacional tras ejecutar el comando de manipulación en la válvula, exponiendo el token final de validación del reto en las cabeceras HTTP.*

4.20.4 Impacto y riesgo

La materialización de una vulnerabilidad que permita el *bypass* de controles en el envío de comandos hacia actuadores físicos representa un impacto catastrófico sobre la Seguridad Física, la Integridad y la Disponibilidad de los servicios industriales de agua potable (*ICS/SCADA*). Permitir que comandos de apertura o cierre de válvulas principales sean ejecutados sin autenticación robusta ni validaciones de seguridad de doble fase abre la puerta a escenarios de sabotaje físico, interrupción masiva del suministro a la población o daños

estructurales severos en las tuberías y estaciones de bombeo debido a golpes de ariete provocados por cierres abruptos.

A nivel de gobernanza y ciberseguridad industrial, este riesgo demuestra que la ausencia de controles transaccionales estrictos en los sistemas de supervisión y adquisición de datos (*SCADA*) puede transformar una brecha de software en una emergencia operativa real que afecte directamente a la comunidad.

4.20.5 Mitigaciones

Para solucionar de manera definitiva la causa raíz de este hallazgo, se deben implementar las siguientes medidas correctivas de cumplimiento obligatorio:

1. **Autenticación Estricta de Doble Factor para Actuadores Críticos:** Exigir una validación física o una confirmación criptográfica de doble factor (2FA) por parte de un operador autorizado antes de permitir la ejecución de cualquier comando que altere el estado de válvulas, bombas o componentes críticos de la red hidráulica.
2. **Implementación de Arquitectura de Seguridad en Capas (*Purdue Model*):** Aislar físicamente los paneles web de gestión (IT) de los controladores lógicos programables (PLCs) y RTUs de campo mediante pasarelas de seguridad unidireccionales y sistemas de detección de anomalías en los comandos de bus industrial.
3. **Validación de Límites Operativos en el Servidor:** Configurar los controladores de campo para que rechacen automáticamente cualquier comando de cambio de estado que viole los parámetros de seguridad predefinidos o que provenga de interfaces no autorizadas para la toma de decisiones críticas.

5. Cadena de Ataque y Correlación de Hallazgos

La correlación de las 20 vulnerabilidades identificadas durante la auditoría de seguridad del entorno **INTERAFAS-LAB** demuestra que los fallos no operan de manera aislada. Por el contrario, configuran una **cadena de ataque (*Kill Chain*) multi-etapa** bien definida, en la cual las vulnerabilidades perimetrales y de configuración inicial facilitan el reconocimiento, habilitan el acceso a componentes lógicos internos y, finalmente, permiten la escalada hacia el entorno de tecnología operacional (*ICS/SCADA*).

Fases de la Cadena de Ataque en INTERAFAS-LAB

Reconocimiento y Exposición Inicial (Fase Externa):

El adversario da inicio a la intrusión aprovechando de manera sistemática la masiva exposición de información sensible mediante archivos de configuración residuales, directorios públicos accesibles y directivas de control mal configuradas en los índices del servidor (**VULN-01**, **VULN-02** y **VULN-04**). Estos artefactos olvidados en producción revelan con precisión quirúrgica las rutas del sistema, notas de despliegue internas y arquitecturas de directorios que reducen drásticamente la necesidad de realizar escaneos ruidosos o a ciegas. Paralelamente, el deficiente manejo e impresión de excepciones en los entornos de ejecución

(**VULN-05**) expone de forma directa rutas absolutas del servidor web y esquemas completos de bases de datos, otorgando al atacante el mapa conceptual indispensable para planificar las siguientes fases del compromiso.

Acceso Inicial y Exfiltración de Datos (Fase de Compromiso Web):

Con la estructura interna expuesta, el atacante explota vulnerabilidades críticas en los formularios y parámetros públicos. Mediante la inyección de consultas en **VULN-07** (Blind SQLi), es capaz de extraer información y credenciales de la base de datos institucional. Asimismo, las fallas de control de acceso tipo IDOR (**VULN-03**, **VULN-11** y **VULN-12**) y el recorrido de directorios (**VULN-09**) permiten saltar los límites de los usuarios ciudadanos, exfiltrando expedientes confidenciales, historiales de servicios y archivos del sistema operativo host como /etc/passwd.

Escalada de Privilegios y Manipulación Transaccional (Fase de Consolidación):

Una vez que el adversario logra establecer un punto de apoyo firme dentro del portal web, procede a consolidar su posición operativa elevando sus privilegios de forma vertical mediante la simple manipulación de las *cookies* de sesión institucionales (**VULN-06**). De manera simultánea y en paralelo, explota complejas fallas de lógica de negocio y debilidades en la validación de parámetros dentro de la pasarela de pagos y el submódulo de vínculos contractuales (**VULN-13** y **VULN-14**), alterando registros financieros oficiales y simulando relaciones contractuales totalmente ficticias pero aceptadas por el sistema. Para asegurar una permanencia a largo plazo, se apoya en vulnerabilidades complementarias como el secuestro de sesiones por fijación (**VULN-10**) y la inyección de contenido almacenado en los reportes ciudadanos (**VULN-08**), garantizando la usurpación silenciosa de las identidades de los operadores legítimos.

Paso de la Red Corporativa (IT) a la Red Industrial (OT):

Esta etapa representa el punto de inflexión más crítico de la intrusión, donde la amenaza cruza la frontera lógica de segmentación entre la red corporativa de tecnología de información y los sistemas hiper-restringidos de control de campo. Aprovechando que los hallazgos perimetrales industriales (**VULN-15** y **VULN-16**) exponen las políticas del *gateway* operacional, el atacante logra burlar de forma trivial los filtros de red mediante la simple falsificación de la cabecera HTTP X-Forwarded-For (*IP Spoofing*), haciéndose pasar por un segmento de confianza interno. A través de este canal de acceso simulado pero exitoso, el intruso interroga de manera automatizada los flujos de telemetría en tiempo real (**VULN-17**), logrando descubrir la topología completa de la infraestructura y las complejas cadenas de dependencia que sostienen a los activos industriales (**VULN-19**).

Compromiso Final y Control de Actuadores (Fase Operacional Crítica):

Al disponer de una visibilidad completa de la red industrial y aprovechando el almacenamiento institucional sumamente inseguro de hashes de contraseñas débiles dentro de los manifiestos públicos de actualización de firmware (**VULN-18**), el atacante procede a descifrar fuera de línea la credencial crítica correspondiente a la Mesa de Soporte OT (Soporte343). Utilizando estas credenciales legítimas combinadas y aprovechando la alarmante ausencia de controles de autenticación multifactor o doble factor en los comandos de campo, se materializa la fase terminal del ataque con el **VULN-20**: el *bypass* absoluto de la validación de estado sobre el controlador de la válvula principal de distribución hídrica (V-SL-201). Esto demuestra de manera irrefutable cómo una concatenación de múltiples y aparentes pequeñas fallas de software en la capa web puede escalar de forma implacable hasta comprometer drásticamente la seguridad física, la integridad y la disponibilidad operativa de toda una infraestructura crítica metropolitana.

6. Calidad del Informe, Conclusiones y Cierre

6.1 Conclusiones Propias de la Auditoría

La auditoría integral realizada sobre el entorno académico e industrial **INTERAFAS-LAB** evidencia que la seguridad de una infraestructura digital no depende únicamente de la robustez de sus componentes individuales, sino de la coherencia y el rigor en el diseño de su arquitectura global. Se constató una desconexión crítica entre los controles de seguridad perimetral corporativa y los sistemas de control industrial (*ICS/SCADA*), donde múltiples fallas de diseño —como la confianza ciega en entradas de usuario, el uso de cabeceras HTTP manipulables y la persistencia de credenciales derivables mediante hashes obsoletos— facilitan la ejecución de cadenas de ataque completas. La remediación de estas vulnerabilidades exige no solo parches puntuales de código, sino una reestructuración profunda basada en el principio de menor privilegio, segmentación física y lógica estricta (*Purdue Model*), y la implementación obligatoria de validaciones robustas en el servidor para proteger tanto los datos ciudadanos como la estabilidad física de los servicios esenciales de la metrópoli.

6.2 Cierre Documental

El presente informe se redactó bajo rigurosos estándares técnicos y profesionales, garantizando la trazabilidad documental de cada hallazgo mediante la correlación exacta con las evidencias gráficas obtenidas en los entornos de laboratorio. Con la integración de este apartado de cadena de ataque y conclusiones, el documento cumple a cabalidad con la estructura académica requerida para la validación y entrega formal del proyecto.